ENTRELIBROS · GRUPO 85

Cómo funciona el frontend

Y cómo implementaremos el backend con Django

Una guía para comprender y construir

Explicación del código actual, diccionario de datos, recorridos completos, contratos HTTP propuestos, patrones State y Strategy, transacciones, pruebas e integración progresiva.

Documento preparado a partir de los archivos reales del proyecto. Los modelos de clases y el DER de FigJam se toman como fuente de verdad; Pen define la presentación. Las simplificaciones del mock y las decisiones todavía pendientes se señalan explícitamente.

Fecha: 12 de octubre de 2026. **Proyecto revisado:** E:/universidad/DAO/tp-dao-G85. El backend versionado parte de 388fd5b; el frontend está en cambios locales todavía no confirmados en Git.

Pensado para leer en orden, consultar durante la programación y preparar la explicación del trabajo práctico. Los ejemplos de backend son propuestas didácticas: no representan código ya agregado al proyecto.

Contenido

01 Cómo leer esta guía y cuál es el estado real 4

02 La arquitectura completa, sin atajos 5

03 Tecnologías y estructura de carpetas 6

04 Qué ocurre cuando se abre la aplicación 7

05 Rutas: pantallas, enlaces y recursos 8

06 Contexto global, mensajes y sede de trabajo 9

07 Diseño, componentes y accesibilidad 10

08 Listados, filtros, paginación y selectores 11

09 Formularios y operaciones: guardar no es transicionar 12

10 La API simulada y sus límites concretos 13

11 Diccionario completo de entidades y relaciones 14

12 Administración: socios, sedes, libros y copias 21

13 Préstamo local, desde el catálogo hasta la devolución 22

14 Préstamo interbibliotecario y trazabilidad del envío 23

15 Reservas: esperar un libro no es prestar una copia 24

16 Todas las máquinas de estado y sus operaciones 25

17 Los cuatro reportes: qué cuentan exactamente 26

18 Qué backend existe y cómo ampliar su estructura 28

19 De TypeScript al ORM sin cambiar el dominio 29

20 Contrato HTTP: lectura, creación y edición 30

21 Catálogo completo de comandos HTTP 32

22 Errores, contratos de campos y respuesta a conflictos 33

23 Implementar Socios: modelo y migración, paso a paso 35

24 Implementar Socios: servicios, serializers y API 36

25 Catálogo y circulación: orden de construcción del backend 38

26 Transacciones y concurrencia: la parte que el mock no garantiza 39

27 State para Remito: clases con comportamiento 40

28 Strategy para la tramitación local e interbibliotecaria 41

29 Autenticación, permisos y separación entre Socio y usuario 42

30 Fechas, zonas horarias y vencimientos 44

31 Integrar el backend de manera progresiva 45

32 Implementar los reportes en Django y conectar sus salidas 46

33 Desarrollo local, variables de entorno y comandos 47

34 Docker y despliegue reproducible 48

35 Plan de pruebas: desde el modelo hasta el navegador 50

36 Hoja de ruta para implementar sin perder el control 52

37 Decisiones abiertas y brechas de integración 53

38 Guía de diagnóstico: problemas frecuentes y dónde mirar 55

39 Glosario para comprender el código 57

40 Mapa de fuentes y referencias para seguir estudiando 58

01 Cómo leer esta guía y cuál es el estado real

EntreLibros administra una red de bibliotecas. El frontend permite explorar el catálogo y recorrer administración, circulación, reservas y logística. En este momento funciona principalmente mediante una API simulada que guarda un conjunto de objetos JSON en el navegador.

El backend todavía no implementa esos procesos completos. En la copia revisada sí existen Django, Django REST Framework, configuración de PostgreSQL, documentación OpenAPI, Docker y el módulo de sedes. Socios, catálogo, circulación, logística y reportes conservan la estructura inicial de sus apps, sin los modelos y casos de uso definitivos.

Categoría	Qué significa en esta guía
Implementado	Existe en el código revisado y puede identificarse en un archivo o función concreta.
Simulado	Existe comportamiento en el navegador, pero no una transacción real compartida en PostgreSQL.
Propuesto	Es un contrato o diseño para escribir después. Se adapta a los modelos existentes y se debe confirmar durante la integración.
Pendiente de decisión	El modelo o la documentación del equipo dejan una regla abierta. Esta guía no la resuelve por su cuenta.

Jerarquía de referencias

1. Clases, DER y máquinas de estado de FigJam: entidades, atributos, relaciones y transiciones del dominio.

2. Backend existente: nombres y comportamiento implementados, especialmente Sede y su API.

3. Tipos de frontend: traducción a snake_case de los modelos, con fechas transportadas como texto.

4. Pen: colores, tipografía, distribución, pantallas y estados visuales. Una pantalla no agrega un atributo de dominio.

5. Mocks y ejemplos: material de demostración. Sus elecciones de interfaz no establecen automáticamente nuevas reglas del producto.

Ejemplo de esta jerarquía

Socio conserva numero_socio, documento, nombre, contacto y estado. No se incorporan apellido, domicilio, teléfono y email como campos separados sólo porque una maqueta de Pen los muestre. Libro conserva sus atributos bibliográficos; la portada es un recurso visual separado.

Qué se verificó y qué no

El frontend pasó la compilación de TypeScript y Vite, 13 pruebas de Vitest y una revisión de catálogo, formulario de socios, temas y navegación móvil. También se construyó y arrancó su contenedor; en la aplicación compilada se recorrió una solicitud local hasta préstamo y devolución de material no apto. Eso valida la demostración, no la concurrencia de PostgreSQL ni la autenticación real.

La ejecución conjunta de todo el Compose no se verificó porque en la copia revisada faltaba el archivo .env que requiere el servicio backend. No se ejecutaron pruebas nuevas de Django para redactar este documento. El endpoint health actual responde un estado fijo y no comprueba la conexión de base de datos.

Código de referencia: frontend/src/domain/models.ts | frontend/src/domain/workflows.test.ts | backend/config/urls.py | backend/apps/common/views.py

02 La arquitectura completa, sin atajos

La aplicación tiene tres lugares con responsabilidades distintas. El navegador muestra y recoge datos. Django recibe peticiones y decide si una operación del dominio es válida. PostgreSQL conserva información compartida y aplica las restricciones estructurales. React no reemplaza a Django, y un objeto TypeScript no reemplaza una tabla.

Pantallas React
Componentes, formularios, filtros, enlaces y mensajes.

↓

Contrato LibraryApi
snapshot, save, command, saveBookAuthors y reset.

↓

Adaptador seleccionado
Mock local actualmente; HTTP cuando existan los endpoints.

↓

Django REST Framework - a implementar por módulo
Vista y serializer convierten HTTP en un caso de uso.

↓

Servicios y dominio - a implementar
Validaciones, patrones y coordinación de transacciones.

↓

ORM y PostgreSQL
Persistencia, relaciones, unicidad y bloqueos concurrentes.

Una misma intención con dos adaptadores

Cuando la pantalla ejecuta api.command("socios", 1, "inhabilitar"), expresa una intención del usuario. El adaptador mock modifica un objeto leído de localStorage. El adaptador HTTP envía un POST a /api/socios/1/inhabilitar/. La pantalla no debería conocer cómo se obtiene la persistencia.

El cliente actual reúne toda la información en snapshot(). Es práctico para la demostración porque cada detalle busca socios, libros y sedes dentro de un solo objeto. Sin embargo, al activar HTTP depende de que todos los recursos existan y devuelvan el formato esperado. Integrar sólo Socios exige una estrategia de carga gradual, explicada más adelante.

> ### La autoridad de las reglas cambia al integrar
>
> El mock ayuda a probar pantallas; el backend debe volver a validar la misma intención con datos actuales. Aunque el botón esté deshabilitado o el navegador diga que un ejemplar está disponible, el servidor vuelve a comprobarlo dentro de la operación real.

Código de referencia: frontend/src/api/contracts.ts | frontend/src/api/index.ts | frontend/src/api/http.ts | frontend/src/api/mock.ts

03 Tecnologías y estructura de carpetas

Tecnología	Versión del proyecto	Responsabilidad
react	19.3.0	Construir componentes y actualizar la interfaz.
react-dom	19.3.0	Montar React en el documento HTML.
react-router-dom	7.18.4	Resolver rutas y navegación del navegador.
typescript	7.0.2	Comprobar tipos al compilar.
vite	8.3.4	Servidor de desarrollo y compilación de producción.
tailwindcss	4.3.3	Utilidades CSS y adaptación de los diseños.
lucide-react	1.55.0	Iconos usados en la navegación y componentes.
vitest	5.0.4	Ejecutar pruebas de los recorridos simulados.
prettier	3.9.10	Dar formato legible y consistente al código.

```
frontend/
  public/brand/          Logos claro y oscuro
  public/assets/        Portadas y letras animadas
  src/app/              Router, contexto y estructura de pantallas
  src/components/       Elementos visuales reutilizables
  src/domain/           Modelos, selectores, reportes y pruebas
  src/api/              Contrato, adaptador mock y adaptador HTTP
  src/data/             Conjunto inicial y referencias de portadas
  src/pages/            Listados, formularios, detalles y operaciones
  src/main.tsx          Punto de entrada
  src/styles.css        Tokens de temas y estilos globales
  vite.config.ts        Puerto, proxy y plugins
  Dockerfile            Construcción y publicación con Nginx
  nginx.conf            Archivos estáticos y proxy de API
```

public se copia a la salida sin convertir sus recursos en entidades. src contiene código importado por la aplicación. dist es el resultado generado por Vite, no el lugar para editar pantallas. node_modules contiene dependencias instaladas, y package-lock.json permite reproducir sus versiones con npm ci.

El proyecto conserva React + TypeScript y Django REST Framework + PostgreSQL. No necesita Prisma, una segunda base de datos ni mover los modelos del dominio a un servicio externo para conectar esta interfaz.

Código de referencia: frontend/package.json | frontend/package-lock.json | frontend/vite.config.ts | docs/definicion-producto.md

04 Qué ocurre cuando se abre la aplicación

1. El navegador pide index.html. El documento define el idioma, los metadatos, las fuentes y el elemento div con id root.

2. main.tsx importa los estilos y monta App dentro de StrictMode. El chequeo adicional de desarrollo de React puede repetir efectos; eso no es una segunda operación de negocio.

3. App monta LibraryProvider y BrowserRouter. Las rutas /acceso y las rutas internas tienen tratamientos visuales distintos.

4. LibraryProvider ejecuta refresh() al iniciar. Se activan loading y error, y se solicita api.snapshot().

5. El adaptador devuelve Database o lanza un error. El contexto actualiza db y termina la carga.

6. Layout muestra navegación y contenido. Si no hay datos durante la primera carga, muestra un estado de carga; ante un fallo ofrece reintentar y, en mock, restablecer ejemplos.

7. La ruta determina qué página se renderiza. / redirige a /catalogo; /inicio muestra el resumen operativo.

React en términos concretos

Un componente es una función que produce elementos de interfaz a partir de datos. Sus props son las entradas que recibe; por ejemplo, EntityList recibe resource. useState mantiene datos que cambian, como query, busy o la selección de un diálogo. Cuando cambia un estado, React vuelve a representar el componente. useEffect coordina efectos externos, como cargar datos o escuchar el tema del sistema.

No todos los estados van a la base de datos. Abrir el menú, mostrar un mensaje o cambiar de página son estados de interfaz. Una solicitud asignada, un remito recibido o la condición física de una copia son datos del dominio, y al integrar deben quedar en PostgreSQL.

Por qué existen los identificadores

Las relaciones se transportan como IDs numéricos. Una solicitud puede guardar socio_id=1 y libro_id=2. Para mostrar nombres, la página consulta los registros relacionados. Esto evita repetir el nombre del socio en cada entidad y permite mantener el modelo normalizado.

Código de referencia: frontend/index.html | frontend/src/main.tsx | frontend/src/app/App.tsx | frontend/src/app/context.tsx

05 Rutas: pantallas, enlaces y recursos

Ruta	Qué muestra
/catalogo	Lista de libros con portadas y disponibilidad.
/catalogo/:id	Ficha bibliográfica y ejemplares del libro.
/catalogo/nuevo y /catalogo/:id/editar	Alta y edición de los atributos de Libro y sus autores.
/socios y /sedes	Listados administrativos con búsqueda y estado.
/socios/:id y /sedes/:id	Datos, operaciones de estado y relaciones relevantes.
/ejemplares	Gestión de copias; admite ?libro_id= como filtro.
/solicitudes	Listado, alta y seguimiento del pedido.
/prestamos y /prestamos/:id	Listado y detalle del préstamo generado por una entrega.
/prestamos/devolucion	Buscador de préstamos abiertos para registrar devolución.
/reservas y /remitos	Listado, alta y detalle con las operaciones del proceso.
/reportes y /reportes/:type	Centro de reportes y cálculo con filtros.
/inicio, /apariencia, /acceso	Resumen, preferencias y acceso de demostración.
/sin-acceso y cualquier ruta no reconocida	Pantallas visuales de acceso denegado o página inexistente.

Los recursos libros, socios, sedes y ejemplares admiten edición. Solicitudes, reservas y remitos se crean y después avanzan mediante operaciones específicas. No existe una pantalla genérica para crear un Préstamo: se genera al entregar una solicitud. Esto preserva la relación de una solicitud con, como máximo, un préstamo.

pathOf("libros") devuelve catalogo. El nombre del recurso y la URL de la pantalla no tienen que ser iguales. Tampoco hay que confundir /catalogo/2, que es una ruta de React, con /api/libros/2/, que sería una ruta HTTP del backend.

Los parámetros de búsqueda permiten abrir un formulario con libro y ejemplar preseleccionados. Seleccionar RAY-0001 desde Rayuela lleva a /solicitudes/nuevo?libro_id=2&ejemplar_id=4. El mock vuelve a validar que esas relaciones sean correctas; el backend deberá hacer lo mismo.

Rutas no equivalen a permisos

/sin-acceso sólo representa una pantalla. No existe protección real de rutas, usuario autenticado ni autorización por objeto en el frontend actual. Ocultar un enlace no impide llamar a la API.

Código de referencia: frontend/src/app/App.tsx | frontend/src/pages/resources.ts | frontend/src/pages/EntityForm.tsx

06 Contexto global, mensajes y sede de trabajo

Dato o función	Qué controla	Dónde persiste
db	El conjunto de entidades cargadas desde el adaptador.	En memoria React; el origen puede ser mock o HTTP.
loading / error	Estado de carga y mensaje de fallo.	En memoria.
refresh()	Vuelve a cargar el snapshot y reemplaza db.	No es una escritura de negocio por sí misma.
theme / setTheme()	Claro, oscuro o sistema.	localStorage: entrelibros:theme.
sedeId / setSedeId()	Sede elegida como contexto de trabajo.	En memoria; vuelve a 1 al iniciar una sesión nueva de página.
toast / notify()	Confirmación visible de una operación.	En memoria; se oculta luego de 4,5 segundos.

useLibrary() es el acceso al contexto. Las pantallas pueden obtener db y refresh sin recibirlos como props a través de todos los componentes. Este mecanismo no es una base de datos ni una sesión del servidor. Tener sedeId=2 no demuestra que un usuario tenga autorización para operar en Sede Norte.

Cambiar la sede superior modifica el contexto de inicio y los valores iniciales de ciertos formularios. Los listados conservan sus propios filtros; no se filtran todos automáticamente por sedeId. Socio no tiene una FK a Sede en el modelo, por lo que su listado no incorpora una pertenencia a sede inventada.

El ciclo de una escritura

1. La pantalla activa busy y limpia el error anterior.

2. Espera api.save(...) o api.command(...).

3. Si la operación tiene éxito, espera refresh() para cargar nuevamente los datos.

4. Notifica el resultado y navega al detalle o cierra el diálogo.

5. Si falla, conserva el formulario, muestra el mensaje y vuelve a permitir la acción.

El refresh actual captura su propio error. Por eso una escritura puede haberse confirmado aunque después falle la recarga; antes de repetir una entrega o un despacho, el frontend real deberá distinguir claramente un error de escritura de un error de actualización de vista. La autoridad de si la operación ocurrió será el servidor.

Código de referencia: frontend/src/app/context.tsx | frontend/src/app/Layout.tsx | frontend/src/pages/EntityForm.tsx | frontend/src/pages/Commands.tsx

07 Diseño, componentes y accesibilidad

Pen define una interfaz de biblioteca sobria: fondo crema, verde institucional, acentos terracota, titulares Lora y texto DM Sans. styles.css representa esas decisiones mediante variables como primary, background, card, border y muted-foreground. Al cambiar data-theme en el elemento html, los mismos componentes toman los colores del otro tema.

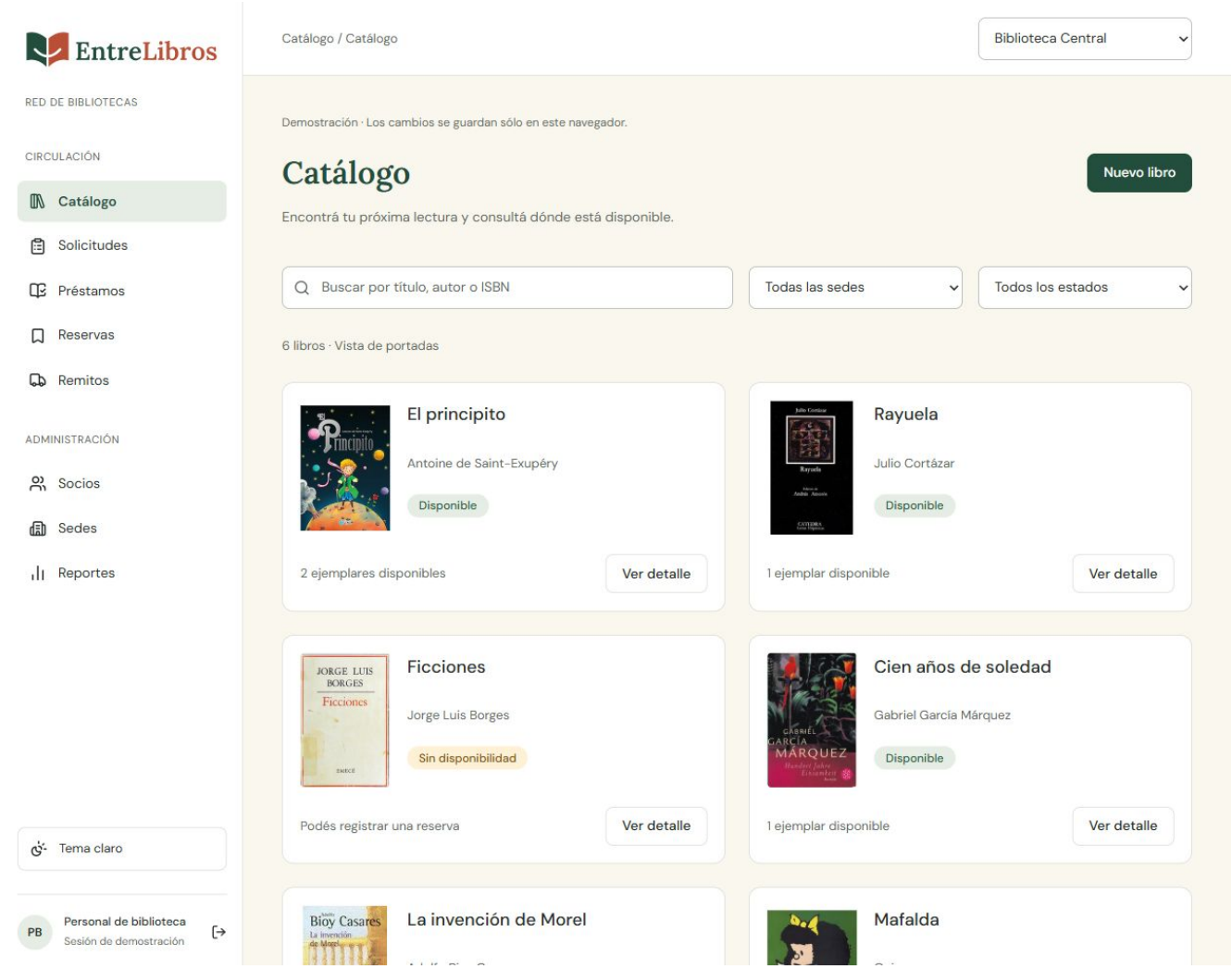


Figura 1. Catálogo implementado en React: navegación, búsqueda, filtros y disponibilidad derivada de las copias. Captura de la aplicación revisada.

Componente	Uso y comportamiento
Button	Acción primaria u outline, estado disabled y texto de carga.
Badge	Traduce códigos del dominio a etiquetas y colores; no cambia estados.
Heading / Back	Título, descripción, acción principal y navegación de retorno.
Field	Asocia etiqueta y campo con un ID; conecta errores con aria-describedby e indica aria-invalid.
SearchBox	Entrada de búsqueda con icono y nombre accesible.

Componente	Uso y comportamiento
Table	Cabeceras y filas; permite desplazamiento horizontal dentro del contenedor.
Empty / Alert	Resultados vacíos, confirmaciones y errores de operación.
ConfirmDialog	Confirmación modal para restablecer datos de ejemplo.
CommandDialog	Formulario modal para una operación del dominio.
Logo / Cover	Assets originales, variantes de tema y alternativa sin portada.

El menú es fijo en escritorio y se oculta en móvil hasta abrirlo. Los listados pueden desbordar dentro de su contenedor horizontal, evitando extender la página completa. Los formularios usan una o dos columnas según el espacio. El enlace Saltar al contenido permite evitar la navegación repetida con teclado.

Los diálogos nativos se abren con showModal(). Mientras una operación está guardando se evita su cierre por la acción de cancelar del diálogo. El acceso incluye letras animadas y un botón para pausar el movimiento; la regla prefers-reduced-motion desactiva la animación cuando el sistema lo pide.

Portadas y límites del diseño

coverByBook relaciona los IDs de seis libros de ejemplo con archivos de public/assets/portadas. No hay un atributo portada en Libro ni una carga de imágenes persistente. Un libro nuevo muestra la alternativa sin portada. Si se decide agregar almacenamiento de portadas, debe tratarse como una decisión explícita, sin alterar silenciosamente el modelo autorizado.

Código de referencia: frontend/src/styles.css | frontend/src/components/ui.tsx | frontend/src/components/Logo.tsx | frontend/src/pages/Lists.tsx | frontend/src/pages/Other.tsx

08 Listados, filtros, paginación y selectores

EntityList arma una representación de filas a partir de Database. Cada fila tiene un ID, un texto preparado para búsqueda, sus sedes vinculadas, un estado y las celdas visuales. La búsqueda se compara sin distinguir mayúsculas. Los filtros de sede y estado trabajan sobre ese conjunto en memoria.

En el catálogo la disponibilidad se calcula a partir de ejemplares. No existe un campo disponible en Libro. Un libro puede tener varias copias: una prestada y otra disponible. authorNames() obtiene los autores recorriendo LibroAutor y Autor; bookName(), socioName() y sedeName() convierten IDs a nombres para la interfaz.

```
export const available = (e: Ejemplar, db: Database) =>
  e.estado === "DISPONIBLE" &&
  e.condicion_fisica === "APTO" &&
  e.sede_actual_id === e.sede_propietaria_id &&
  db.sedes.some((s) => s.id === e.sede_propietaria_id && s.estado === "ACTIVA");
export const authorNames = (book: Libro, db: Database) =>
  db.libro_autores
```

Código de referencia: frontend/src/domain/selectors.ts, líneas 35-41

La condición de sede actual igual a propietaria es una restricción presente en la demostración. El modelo distingue propiedad y ubicación, y deja la política de retorno por confirmar. No debe transformarse esta condición del helper en una regla nueva e incuestionable del dominio. El backend debe acordar la política y, si cambia el cálculo, el selector y sus pruebas deben acompañar ese cambio.

Paginación actual y evolución

Los listados tabulares muestran veinte filas por página cuando hay más resultados. El catálogo muestra todos los libros filtrados. En mock esto es paginación de presentación: todos los registros ya están en memoria. En HTTP list() descarga todas las páginas de DRF, las concatena y después React vuelve a paginar.

Para una base pequeña se puede mantener esa estrategia durante la integración. Para muchos datos conviene que cada pantalla solicite page, search, estado y sede al backend; así el navegador recibe sólo lo necesario. Eso requerirá cambiar la interfaz de lectura del adaptador y el estado de la página, no las entidades del modelo.

Códigos visibles e IDs reales

code("SOL", 42) produce SOL-0042 para mostrar una solicitud. No agrega numero_solicitud al modelo. Los prefijos PRE y RES también son presentación derivada del ID. Remito.numero, Socio.numero_socio y Ejemplar.codigo_inventario sí son atributos reales del dominio y deben persistirse y tener la unicidad definida.

Código de referencia: frontend/src/pages/Lists.tsx | frontend/src/domain/selectors.ts | frontend/src/api/http.ts

09 Formularios y operaciones: guardar no es transicionar

EntityForm representa altas y modificaciones simples. fieldsFor() define qué campos tiene cada recurso, sus etiquetas y opciones. El estado de un socio o sede no se edita con una selección libre: se cambia desde su ficha mediante una operación. Un estado de circulación tampoco se modifica con un PATCH arbitrario.

Las relaciones de los formularios se seleccionan como valores de texto del HTML, y al construir el payload se convierten en números. Una relación opcional vacía pasa a null. El nombre de campo del JSON debe coincidir con el serializer del backend: documento no es dni, y sede_propietaria_id no es sede_id.

Libro y autores

El formulario recibe autores como texto separado por comas. Ese texto se separa y se envía a saveBookAuthors(); no es un campo directo de Libro. Hoy hay dos escrituras: guardar libro y actualizar sus relaciones de autor. Si falla la segunda, el libro podría existir sin que sus autores se hayan guardado. El backend y el adaptador deben definir una operación atómica conjunta o una recuperación explícita.

CommandDialog

El diálogo recibe resource, id y action. Según la acción, pide un ejemplar, fechas, condición física, responsable u observación. Al confirmar obtiene un FormData del formulario, normaliza los IDs y construye fechas con offset -03:00. Esto preserva el valor de los controles datetime-local al enviar.

```
const formData = new FormData(e.currentTarget);
for (const f of fields) {
  const raw = String(formData.get(f.key) ?? "");
  input[f.key] = f.key.endsWith("_id")
    ? Number(raw)
    : f.type === "datetime-local"
      ? `${raw}${raw.length === 16 ? ":00" : ""}-03:00`
      : raw;
}
if (action === "recibir") input.verificados = checked;
await api.command(resource, id, action, input);
```

Código de referencia: frontend/src/pages/Commands.tsx, líneas 161-171

Para entregar no se aplica automáticamente un plazo de catorce días. La fecha inicial sugerida es el día actual a las 12:00, una comodidad de interfaz; el operador indica el vencimiento. Para preparar una reserva también se exige un límite futuro ingresado explícitamente. La guía no declara esos valores iniciales como política de préstamos.

El atributo required del HTML ayuda a evitar campos vacíos, y busy evita clics repetidos mientras se espera una respuesta. Ninguno protege contra otra pestaña ni sustituye la validación del servidor. El backend debe ignorar o rechazar cambios directos a campos controlados y volver a comprobar la operación completa.

Código de referencia: frontend/src/pages/EntityForm.tsx | frontend/src/pages/resources.ts | frontend/src/pages/Commands.tsx | frontend/src/api/contracts.ts

10 La API simulada y sus límites concretos

createMockApi() recibe un almacenamiento compatible con getItem, setItem y removeItem. En el navegador usa localStorage. La clave entrelibros:demo:v1 contiene un JSON con los arrays de Database. Si no hay información guardada, devuelve una copia del conjunto seed; después de una escritura, conserva el conjunto completo.

1. read() carga y deserializa el JSON, o copia los datos de ejemplo.

2. saveEntity() valida y crea o actualiza un registro administrativo o inicializa un proceso.

3. applyCommand() ejecuta una operación y puede modificar varias entidades relacionadas.

4. write() guarda el conjunto solamente si la operación terminó sin lanzar un error.

5. snapshot() permite que React lea un conjunto nuevo después del guardado.

La secuencia evita persistir una operación que falla en ese mismo intento. No equivale a una transacción SQL: dos pestañas pueden leer el mismo JSON, escribir cambios distintos y perder una modificación. nextId() calcula máximo más uno, apropiado para ejemplos, pero no reemplaza una PK de PostgreSQL.

Qué controla hoy

- Unicidad simulada para datos como documento, número de socio, nombre de sede, ISBN, código de inventario y número de remito.
- Socios habilitados y sedes activas al iniciar operaciones.
- Asignación de una copia que corresponda al libro y origen y se encuentre disponible.

- Entrega posterior a preparación o recepción, con vencimiento posterior a entrega.
- Devolución de préstamo abierto y registro de condición física.
- Transiciones de remito y creación de eventos; reserva con plazo de retiro explícito.

Qué no debemos trasladar literalmente

El mock centraliza muchas ramas en applyCommand(). Eso permite probar el frontend, pero no demuestra State ni Strategy como clases. La comparación de documentos y números no contiene una política definitiva de normalización. La recepción copia la condición física existente al detalle y no registra una inspección individual editable. La reserva asigna sólo ejemplares disponibles en su sede de retiro, sin gestionar traslado desde otra sede.

La expiración de reservas es una acción manual, no una tarea periódica. No hay orden FIFO automático ni atención de la siguiente reserva al liberar una copia. No se autoriza al responsable escrito en el formulario: es un texto de demostración. Estos límites se deben resolver o mantener explícitos durante la implementación real.

Código de referencia: frontend/src/api/mock.ts | frontend/src/data/seed.ts | frontend/src/domain/models.ts

11 Diccionario completo de entidades y relaciones

Este diccionario se genera a partir de las interfaces reales de frontend/src/domain/models.ts. Cada campo conserva su nombre de transporte. Las notas explican cómo se corresponde con el DER y qué deberá preservar el ORM. Una interface TypeScript sólo se verifica al compilar: no valida por sí misma un JSON recibido del servidor.

number representa los identificadores numéricos en el frontend. string representa texto y también las fechas ISO 8601 del transporte. null significa ausencia admitida; no se reemplaza por 0, una cadena vacía o una fecha inventada. Los tipos Estado... representan conjuntos de códigos, pero PostgreSQL también deberá restringir los valores válidos.

Sede

Una biblioteca de la red. No se confunde con una ubicación temporal de una copia.

Campo JSON / TypeScript	Tipo actual	Significado y uso
id	number	Identificador interno del registro. Se genera en el servidor; en mock se calcula máximo más uno.
nombre	string	Nombre mostrado al usuario. En Socio es el nombre completo del atributo definido.
direccion	string	Dirección de la sede; no representa el domicilio del socio.
contacto	string	Texto de contacto según el modelo; no supone campos separados email y teléfono.
horarios	string	Horario de atención inicialmente expresado como texto.

Campo JSON / TypeScript	Tipo actual	Significado y uso
estado	"ACTIVA" | "BAJA"	Código de estado de la entidad; las operaciones habilitadas dependen de él.

Ya tiene modelo Django y migración. Su nombre tiene unicidad sin distinguir mayúsculas. Estado inicial ACTIVA; baja lógica BAJA y reactivación. Las comprobaciones sobre procesos en curso están pendientes.

Socio

La persona que puede solicitar material. No es necesariamente un usuario de acceso al sistema.

Campo JSON / TypeScript	Tipo actual	Significado y uso
id	number	Identificador interno del registro. Se genera en el servidor; en mock se calcula máximo más uno.
numero_socio	string	Identificador de socio de la red. Atributo real y único, distinto del id.
documento	string	Identificación documental como texto. Atributo único; normalización por acordar.
nombre	string	Nombre mostrado al usuario. En Socio es el nombre completo del atributo definido.
contacto	string	Texto de contacto según el modelo; no supone campos separados email y teléfono.
estado	"HABILITADO" | "INHABILITADO"	Código de estado de la entidad; las operaciones habilitadas dependen de él.

numero_socio y documento son únicos. El estado inicial del diagrama es HABILITADO. Inhabilitar impide nuevos préstamos, pero no debe impedir devolver material existente.

Autor

Una entidad bibliográfica que puede participar en varios libros.

Campo JSON / TypeScript	Tipo actual	Significado y uso
id	number	Identificador interno del registro. Se genera en el servidor; en mock se calcula máximo más uno.
nombre	string	Nombre mostrado al usuario. En Socio es el nombre completo del atributo definido.

Su identificación se comparte entre libros. El formulario de demostración busca coincidencias por nombre; la política de deduplicación de autores no queda definida por esa comodidad del mock.

Libro

La ficha bibliográfica de una edición, distinta de sus copias físicas.

Campo JSON / TypeScript	Tipo actual	Significado y uso
id	number	Identificador interno del registro. Se genera en el servidor; en mock se calcula máximo más uno.
isbn	string | null	Identificación bibliográfica opcional. null si no existe. Unicidad cuando está informado.
titulo	string	Título bibliográfico de la edición registrada.
editorial	string	Editorial indicada en la ficha.
edicion	string	Identificación de edición del libro.

ISBN es único cuando existe y admite null. Los autores se relacionan mediante LibroAutor. No tiene máquina de estados inicial, ni disponibilidad o portada como atributos del modelo revisado.

LibroAutor

La relación de muchos a muchos entre Libro y Autor.

Campo JSON / TypeScript	Tipo actual	Significado y uso
libro_id	number	FK a Libro; su valor debe corresponder a un libro existente.
autor_id	number	FK a Autor; junto con libro_id define una asociación única.

El par libro_id + autor_id identifica una asociación y no se repite. No se transporta un id artificial de esa asociación. La representación técnica del vínculo en Django deberá preservar esa unicidad.

Ejemplar

Una copia física identificable que pertenece a un libro y a una sede propietaria.

Campo JSON / TypeScript	Tipo actual	Significado y uso
id	number	Identificador interno del registro. Se genera en el servidor; en mock se calcula máximo más uno.
codigo_inventario	string	Identificación única de una copia física. No es el ISBN del libro.
libro_id	number	FK a Libro; su valor debe corresponder a un libro existente.
sede_propietaria_id	number	FK a Sede propietaria; permanece al prestar o trasladar la copia.
sede_actual_id	number | null	FK opcional a Sede de ubicación actual. null cuando la copia está en tránsito.

Campo JSON / TypeScript	Tipo actual	Significado y uso
estado	EstadoEjemplar	Código de estado de la entidad; las operaciones habilitadas dependen de él.
condicion_fisica	string	Condición física del material. La demostración usa APTO / NO_APTO.

codigo_inventario es único. La sede actual puede ser null en tránsito; la propietaria no desaparece. Disponibilidad y condición física se representan por separado. Una copia no admite dos asignaciones o préstamos vigentes.

SolicitudPrestamo

El pedido que vincula socio, libro, origen y sede donde se entrega.

Campo JSON / TypeScript	Tipo actual	Significado y uso
id	number	Identificador interno del registro. Se genera en el servidor; en mock se calcula máximo más uno.
socio_id	number	FK a Socio. Habilitación necesaria según la operación, no por ser una FK.
libro_id	number	FK a Libro; su valor debe corresponder a un libro existente.
ejemplar_id	number | null	FK a Ejemplar. Opcional hasta asignación en Solicitud; obligatoria en DetalleRemito.
sede_origen_id	number	FK a Sede de origen. En Solicitud debe corresponder a la propietaria del ejemplar.
sede_destino_id	number	FK a Sede de entrega o destino del remito, según la entidad.
reserva_id	number | null	FK opcional y única a Reserva. Conserva la asociación del pedido con su espera.
fecha_solicitud	string	Momento de registro de la solicitud; en el backend lo establece el servidor.
tipo	"LOCAL" | "INTERBIBLIOTECARIO"	Circuito LOCAL o INTERBIBLIOTECARIO, coherente con origen y destino.
estado	EstadoSolicitud	Código de estado de la entidad; las operaciones habilitadas dependen de él.

La copia puede ser null hasta asignarse. Una reserva se relaciona como máximo con una solicitud, y cada solicitud produce como máximo un préstamo. LOCAL exige origen=destino; INTERBIBLIOTECARIO exige que sean distintos.

Prestamo

La entrega efectiva al socio y su obligación de devolución.

Campo JSON / TypeScript	Tipo actual	Significado y uso
id	number	Identificador interno del registro. Se genera en el servidor; en mock se calcula máximo más uno.
solicitud_id	number	FK a SolicitudPrestamo. Única en Prestamo; opcional en DetalleRemito.
fecha_entrega	string	Momento de entrega que inicia el préstamo. No es la fecha del envío.
fecha_vencimiento	string	Fecha/hora acordada para devolver; debe ser posterior a la entrega.
fecha_devolucion	string | null	Momento de devolución. null indica que todavía no se registró.
condicion_devolucion	string | null	Condición del ejemplar al devolver. null hasta registrar devolución.

solicitud_id es único. No guarda un estado VENCIDO ni duplica socio y ejemplar: se obtienen de la solicitud cumplida e inmutable. El vencimiento se deriva de fechas y de si existe devolución.

Reserva

La espera de un socio por un libro y una sede de retiro.

Campo JSON / TypeScript	Tipo actual	Significado y uso
id	number	Identificador interno del registro. Se genera en el servidor; en mock se calcula máximo más uno.
socio_id	number	FK a Socio. Habilitación necesaria según la operación, no por ser una FK.
libro_id	number	FK a Libro; su valor debe corresponder a un libro existente.
sede_retiro_id	number	FK a Sede donde el socio retirará el material reservado.
ejemplar_asignado_id	number | null	FK opcional a la copia asignada a una reserva.
fecha_reserva	string	Momento en que se registra la espera.
fecha_limite_retiro	string | null	Límite opcional hasta preparar una asignación. Su plazo no está automatizado.
estado	EstadoReserva	Código de estado de la entidad; las operaciones habilitadas dependen de él.

La copia y el plazo pueden ser null al registrar la reserva. Evitar duplicados vigentes por socio, libro y sede. El modelo no establece una duración automática ni el orden de la cola.

Remito

El envío entre dos sedes; agrupa detalles y eventos de trazabilidad.

Campo JSON / TypeScript	Tipo actual	Significado y uso
id	number	Identificador interno del registro. Se genera en el servidor; en mock se calcula máximo más uno.
numero	string	Número único del remito, persistido y visible en la interfaz.
sede_origen_id	number	FK a Sede de origen. En Solicitud debe corresponder a la propietaria del ejemplar.
sede_destino_id	number	FK a Sede de entrega o destino del remito, según la entidad.
estado	EstadoRemito	Código de estado de la entidad; las operaciones habilitadas dependen de él.
fecha_creacion	string	Momento de registro del remito y de su evento inicial.
fecha_despacho	string | null	Momento en que se despacha el envío. null antes de despacharlo.
fecha_recepcion	string | null	Momento de recepción. null mientras no se recibió.

numero es único. Puede estar vacío en preparación; empaquetar o despachar exige contenido. La fecha de despacho y la de recepción se establecen en sus operaciones. Las transiciones se implementarán con State.

DetalleRemito

Una copia incluida en un remito con sus condiciones de salida y recepción.

Campo JSON / TypeScript	Tipo actual	Significado y uso
id	number	Identificador interno del registro. Se genera en el servidor; en mock se calcula máximo más uno.
remito_id	number	FK al remito al que pertenece el detalle o el evento.
ejemplar_id	number	FK a Ejemplar. Opcional hasta asignación en Solicitud; obligatoria en DetalleRemito.
solicitud_id	number | null	FK a SolicitudPrestamo. Única en Prestamo; opcional en DetalleRemito.
condicion_salida	string	Condición registrada cuando se incluye o prepara el envío de la copia.
condicion_recepcion	string | null	Condición observada al recibir. null antes de la recepción.

El par remito_id + ejemplar_id es único. solicitud_id puede ser null, por ejemplo para un traslado no vinculado a una solicitud. No se crea una FK obligatoria que impida registrar esos traslados.

EventoRemito

Un registro ordenado de una transición del envío.

Campo JSON / TypeScript	Tipo actual	Significado y uso
id	number	Identificador interno del registro. Se genera en el servidor; en mock se calcula máximo más uno.
remito_id	number	FK al remito al que pertenece el detalle o el evento.
fecha_hora	string	Instante de la transición de trazabilidad, asignado por el servidor.
estado_anterior	EstadoRemito | null	Código anterior del remito, o null en el evento inicial.
estado_nuevo	EstadoRemito	Código al que pasó el remito en esa transición.
responsable	string	Responsable del evento. El backend deberá relacionarlo con el actor autorizado sin cambiar el modelo por iniciativa de la interfaz.
observacion	string	Texto contextual del evento; no sustituye estado ni fecha.

Cada remito tiene al menos su evento inicial. estado_anterior es null en la creación. El historial se agrega y se consulta; no se edita libremente para reescribir el trayecto.

Código de referencia: frontend/src/domain/models.ts | backend/apps/sedes/models.py | FigJam: Modelos, clases y DER

Relaciones que deben mantenerse

Relación	Cardinalidad / integridad
Libro - Ejemplar	Un libro tiene cero o más copias; cada ejemplar pertenece a un libro.
Sede - Ejemplar	Una propietaria por copia. Ubicación actual distinta y opcional en tránsito.
Libro - Autor	Muchos a muchos por LibroAutor; no repetir el par.
Socio - Solicitud	Un socio por solicitud; un socio puede tener muchas solicitudes.
Solicitud - Ejemplar	Cero o una copia hasta asignación; una vez entregada mantiene su identidad.
Solicitud - Préstamo	Una solicitud tiene cero o un préstamo; cada préstamo corresponde a una solicitud.
Reserva - Solicitud	Cero o una relación por cada lado; impedir reutilizar una misma reserva para varios pedidos.
Remito - Detalle	Cero o más detalles en preparación. Cada detalle corresponde a un remito.
Remito - Evento	Uno o más eventos, empezando por la creación.
Detalle - Solicitud	Cero o una solicitud por detalle; los traslados sin solicitud también se representan.

12 Administración: socios, sedes, libros y copias

Socios: alta, consulta, edición y habilitación

El listado busca por nombre, documento, número de socio y también el contacto incluido en su texto de búsqueda. La ficha muestra sus atributos y las solicitudes vinculadas. El alta usa exactamente los cuatro campos editables del modelo y comienza habilitada. Habilitar e inhabilitar son comandos separados de editar datos personales.

El backend de Socios deberá reproducir la unicidad de numero_socio y documento mediante restricciones de base de datos, además de la validación amigable del serializer. Una comprobación exists() previa mejora el mensaje, pero no impide dos inserciones simultáneas; la restricción persistente es la protección final.

Sedes: reutilizar lo existente

Sede ya está implementada en Django. Sus datos se editan por API y el estado se controla por dar-de-baja y reactivar. El DELETE no está permitido. Antes de ampliar el módulo hay que conservar su nombre único sin distinguir mayúsculas, el estado read-only y los servicios que bloquean la fila al cambiar estado.

La baja de una sede con operaciones en curso requiere una política que los diagramas dejan pendiente. La implementación actual cambia su estado sin comprobar préstamos, reservas o remitos activos. La regla confirmada sí exige impedir iniciar operaciones nuevas desde o hacia una sede de baja; no equivale a cancelar automáticamente su historial.

Catálogo: ficha bibliográfica y autores

Libro almacena título, ISBN opcional, editorial y edición. Autor y LibroAutor mantienen los nombres asociados. El frontend puede crear y editar una ficha, pero todavía no tiene una página independiente de administración de autores. Los seis libros iniciales tienen portadas ya disponibles; nuevos libros usan la alternativa sin imagen.

Para ISBN ausente, no se debe inventar una deduplicación por título solamente: dos ediciones pueden compartir un título. El criterio de identificación sin ISBN sigue pendiente. La base deberá impedir repetir ISBN informado y el par LibroAutor, mientras el proceso de negocio confirma qué otras coincidencias deben advertirse.

Ejemplares: una copia no se transforma en otra

El alta define código de inventario, libro, sede propietaria y condición física. En la demostración la ubicación inicial coincide con la propietaria. La edición no cambia el libro ni la propietaria de una copia ya existente y se limita a copias no comprometidas por una operación. El estado no se selecciona libremente en el formulario.

Después de una devolución, la copia queda EN_REVISION. Si se informa condición APTO y está en la sede admitida por la política, una aprobación la vuelve disponible. Una copia NO_APTO no se presta por el hecho de cerrar un préstamo. Dar de baja conserva identidad e historial.

Una edición administrativa puede impactar circulación

Modificar contacto es distinto de reemplazar un ejemplar, cambiar propietario o cambiar un estado. El backend debe permitir cambios de datos autorizados y proteger las relaciones e invariantes del proceso, incluso desde Django Admin.

Código de referencia: frontend/src/pages/resources.ts | frontend/src/pages/Details.tsx | frontend/src/api/mock.ts | backend/apps/sedes/services.py

13 Préstamo local, desde el catálogo hasta la devolución

El circuito local mantiene iguales las sedes de origen y entrega. La entrega inicia el préstamo. La creación de una solicitud o su asignación no consumen automáticamente el plazo de lectura. Los cambios a Solicitud, Ejemplar y Prestamo deben conservarse juntos cuando se implementen en Django.

Paso	Acción del usuario	Cambio del dominio
1	Abrir un libro y elegir copia o crear solicitud.	Solicitud con socio, libro y sedes; ejemplar opcional.
2	Registrar la solicitud.	PENDIENTE si no asignó copia; ASIGNADA si se valida la copia seleccionada.
3	Asignar ejemplar si el pedido está pendiente.	Solicitud ASIGNADA y Ejemplar ASIGNADO; el origen coincide con la propietaria.
4	Preparar entrega local.	Solicitud LISTA_PARA_ENTREGA y Ejemplar LISTO_PARA_ENTREGA.
5	Confirmar fecha de entrega y vencimiento.	Se crea un Prestamo único; Solicitud CUMPLIDA y Ejemplar PRESTADO.
6	Registrar devolución y condición.	Se completan fecha_devolucion y condicion_devolucion; Ejemplar EN_REVISION.
7	Revisar la copia y aprobar disponibilidad cuando corresponda.	Sólo una copia apta y admisible vuelve a DISPONIBLE.

Ejemplo con los datos iniciales

Rayuela es Libro.id=2 y RAY-0001 es Ejemplar.id=4, perteneciente a Biblioteca Central.id=1. Seleccionar esa copia abre el formulario con ambos IDs preseleccionados. Al elegir Lucía Fernández.id=1 y retirar en Central, el circuito es LOCAL. El backend debe calcular o validar el tipo; no confiar en una etiqueta enviada por React.

Validaciones de cada momento

- Alta y asignación: socio habilitado, sedes activas, libro existente, copia disponible y correspondencia de libro y sede propietaria.

- Preparación: solicitud local asignada y copia en la situación esperada; no cualquier solicitud interbibliotecaria.
- Entrega: pedido listo, material en la sede de entrega, condición apta, socio habilitado, fechas coherentes y ausencia de préstamo previo para esa solicitud.
- Devolución: préstamo abierto, fecha no anterior a entrega y condición registrada. Un socio inhabilitado sigue pudiendo devolver.
- Aprobación de disponibilidad: revisión terminada, condición apta y política de ubicación/retorno aplicada.

Qué debería ocurrir ante una carrera

Si dos empleados intentan asignar RAY-0001 al mismo tiempo, no deben crearse dos solicitudes con asignación vigente. Si dos empleados entregan SOL-0043, no deben crearse dos préstamos. El backend relee y bloquea los registros dentro del caso de uso, y PostgreSQL aplica las restricciones que sean expresables. El navegador recibe un error de negocio cuando el estado ya cambió.

Código de referencia: frontend/src/api/mock.ts: saveEntity y applyCommand | frontend/src/pages/Commands.tsx | frontend/src/domain/workflows.test.ts

14 Préstamo interbibliotecario y trazabilidad del envío

Este circuito conserva distinta la sede de origen propietaria y la sede de entrega. No cambia la pertenencia de la copia cuando viaja. La ubicación puede ser null durante el tránsito, y una recepción en destino no inicia por sí misma el préstamo al socio.

1. Crear y asignar la solicitud con origen y destino distintos. La estrategia interbibliotecaria valida el pedido y evita reutilizar una copia comprometida.

2. Preparar un remito con ese origen y destino, y agregar la copia asignada. Su DetalleRemito conserva la solicitud vinculada y la condición de salida.

3. Empaquetar un envío no vacío. La copia no se vuelve disponible por estar en un paquete.

4. Despachar: registrar fecha, pasar la solicitud a EN_TRASLADO, la copia a EN_TRANSITO y su ubicación actual a null.

5. Registrar salida: Remito pasa de DESPACHADO a EN_TRANSITO, agregando un evento propio.

6. Recibir en destino: verificar el contenido, registrar responsable y observación, fecha de recepción y ubicación de las copias.

7. Si corresponde a una solicitud apta y verificada, preparar la entrega: Solicitud LISTA_PARA_ENTREGA y Ejemplar LISTO_PARA_ENTREGA.

8. Entregar al socio: generar el préstamo único con sus fechas, igual que en el circuito local.

9. Devolver y aplicar la política de revisión y retorno que el equipo confirme. No se inventa un retorno automático al registrar devolución.

Estado del remito y estado de la solicitud son distintos

Un remito agrupa copias y puede incluir varias solicitudes o detalles sin solicitud. Remito.RECIBIDO describe el transporte; Solicitud.CUMPLIDA describe la entrega al socio. Si el envío se recibe a la mañana y el socio retira a la tarde, son dos operaciones y dos instantes distintos. No se debe cerrar el préstamo ni empezar su plazo con la fecha de despacho.

Lectura de la ficha y del historial

La ficha del remito muestra las copias, libros, solicitudes, condiciones y eventos. Los eventos se ordenan por fecha_hora y conservan estado anterior, nuevo, responsable y observación. La ficha de la solicitud enlaza sus remitos y el préstamo que finalmente genere. No inventa un historial completo de solicitud en una tabla que no existe en el modelo.

Alcance de la simulación de recepción

El mock exige verificar todo el contenido; la recepción parcial no se implementó. Su condicion_recepcion se copia de la condición física conocida, sin un formulario individual de inspección para cada detalle. En el backend se deberá definir cómo registrar la condición observada y gestionar material no apto antes de considerarlo listo para entregar. No basta con copiar el mismo valor en todos los casos reales.

Código de referencia: frontend/src/pages/Details.tsx | frontend/src/pages/Commands.tsx | frontend/src/api/mock.ts: rama remitos

15 Reservas: esperar un libro no es prestar una copia

Reserva vincula Socio, Libro y Sede de retiro. Su ejemplar_asignado_id y fecha_limite_retiro comienzan en null. No equivale a un préstamo activo y no debe registrar por adelantado una fecha de entrega del material.

Estado	Operación del recorrido	Efecto principal
PENDIENTE	Registrar reserva; después asignar y preparar.	Espera sin copia; evita duplicados vigentes.
LISTA_PARA_RETIRO	Asignación con fecha límite explícita.	Copia preparada y solicitud relacionada para entregar.
CUMPLIDA	Entregar al socio.	Se genera préstamo por su solicitud y se cierra la reserva.
CANCELADA	Cancelar reserva pendiente o lista.	Libera una asignación y cancela su solicitud no cumplida cuando corresponda.
EXPIRADA	Registrar vencimiento del plazo de retiro.	Sólo si el límite ya pasó; libera material según las reglas de la operación.

La simulación de asignación busca una copia disponible en la misma sede de retiro. Luego crea la solicitud vinculada, la deja LISTA_PARA_ENTREGA y marca la reserva LISTA_PARA_RETIRO. Esta automatización conecta las pantallas, pero la tramitación de una reserva que necesite traslado desde otra sede no está resuelta.

Lo que se debe acordar antes de programar la cola

- Orden de atención entre reservas elegibles: el diagrama no impone todavía FIFO ni prioridades.
- Duración del plazo de retiro: se ingresa explícitamente y no hay valor automático confirmado.
- Momento y mecanismo de expiración: acción explícita, comando de mantenimiento o tarea programada.
- Tratamiento de una copia liberada: atender la siguiente reserva exige una selección coherente y concurrente.
- Tratamiento de reservas con sede dada de baja y procesos ya iniciados.

Las restricciones del backend pueden impedir dos reservas vigentes del mismo socio por el mismo libro y sede. Sin embargo, una restricción de unicidad total también impediría una reserva nueva después de que otra se cumplió o canceló. La protección correcta debe limitarse a los estados vigentes, además de bloquear el recurso compartido cuando se asigne una copia.

> **Estado exacto del modelo**
>
> El código de reserva lista es LISTA_PARA_RETIRO. No se agrega ASIGNADA como otro estado de Reserva sólo porque una maqueta diga Ejemplar asignado. ASIGNADA pertenece a SolicitudPrestamo y ASIGNADO a Ejemplar.

Código de referencia: frontend/src/domain/models.ts: EstadoReserva | frontend/src/api/mock.ts: rama reservas | frontend/src/pages/Details.tsx

16 Todas las máquinas de estado y sus operaciones

Los códigos en mayúsculas son la representación de transporte del modelo. Las etiquetas legibles de la interfaz son traducciones. Una transición requiere validar el estado previo y sus condiciones; no se acepta que el cliente indique directamente el estado siguiente para saltar pasos.

Entidad	Recorrido principal	Desvíos y límites
Sede	ACTIVA -> BAJA -> ACTIVA	Baja lógica mediante operaciones; no borrado físico.
Socio	HABILITADO -> INHABILITADO -> HABILITADO	Inhabilitar no cancela devoluciones existentes.
Solicitud	PENDIENTE -> ASIGNADA -> LISTA_PARA_ENTREGA -> CUMPLIDA	Interbibliotecaria: ASIGNADA -> EN_TRASLADO -> LISTA_PARA_ENTREGA. RECHAZADA y CANCELADA según validación y reglas.
Remito	EN_PREPARACION -> EMPAQUETADO -> DESPACHADO -> EN_TRANSITO -> RECIBIDO	CANCELADO sólo desde preparación o empaquetado; no desde despacho.
Reserva	PENDIENTE -> LISTA_PARA_RETIRO -> CUMPLIDA	CANCELADA desde pendiente/lista; EXPIRADA al vencer plazo de retiro.

Entidad	Recorrido principal	Desvíos y límites
Ejemplar	DISPONIBLE -> ASIGNADO -> LISTO_PARA_ENTREGA -> PRESTADO -> EN_REVISION	Interbibliotecario incluye EN_TRANSITO antes de listo. Revisión apta permite disponibilidad; BAJA conserva historial.
Préstamo	Activo derivado -> devuelto por fecha_devolucion	Vencido se deriva de fecha y ausencia de devolución; no es un campo estado persistido.

Transiciones que el frontend todavía no cubre por completo

RECHAZADA está representado como estado de solicitud, pero no hay una acción independiente de rechazo en Command. El diagrama admite cancelación después de quedar lista para entrega gestionando el ejemplar; la simulación permite cancelar sólo solicitudes PENDIENTE o ASIGNADA y exige definir el tratamiento posterior. Estos casos deben especificarse e implementarse, no ocultarse con un PATCH libre.

La máquina de Ejemplar incluye retorno y revisión; su implementación completa depende de la política de devolución/retorno. El frontend tampoco implementa atención automática de la siguiente reserva cuando se cancela o expira otra. La representación de tipos y pantallas está disponible, pero todavía no existe toda la lógica del backend.

Invariantes frente a etiquetas

Una etiqueta verde Disponible no es una autorización de préstamo. Una operación puede rechazarse si cambió el estado, se inhabilitó el socio o se dio de baja una sede desde otra sesión. Cada caso de uso debe aceptar una intención, releer los datos y evaluar sus invariantes; el cliente actualizará su vista con el resultado.

Código de referencia: frontend/src/domain/models.ts | frontend/src/domain/selectors.ts: statusLabels | frontend/src/api/contracts.ts: Command | frontend/src/api/mock.ts

17 Los cuatro reportes: qué cuentan exactamente

Los reportes actuales se calculan en el navegador mediante buildReport(). ReportPage separa el borrador de filtros de los filtros aplicados: cambiar una fecha no recalcula el reporte hasta presionar Aplicar. Los resultados pueden exportarse en CSV con encabezados, separador punto y coma y marcador de codificación para facilitar su lectura.

RP01 - Préstamos vencidos

Al corte del final del día indicado, incluye préstamos ya entregados cuyo vencimiento es anterior al corte y cuya devolución es nula o posterior al corte. Obtiene el socio, el libro y la sede de entrega desde la solicitud. Ordena el detalle por sede; el resumen actual es la cantidad total, no una tabla adicional de subtotales por sede.

daysLate() calcula diferencia de días de calendario en Buenos Aires. Un préstamo con vencimiento el 6 de octubre y corte el 9 arroja tres días. Si se devolvió el 10, sigue figurando vencido en el corte histórico del 9. Un préstamo vencido hace horas dentro del mismo día puede aparecer con cero días de atraso: vencido y cantidad de días no son exactamente la misma comparación.

RP02 - Demanda interbibliotecaria

Incluye sólo solicitudes INTERBIBLIOTECARIO por fecha_solicitud, y excluye RECHAZADA y CANCELADA. Agrupa por libro y sede_destino como sede solicitante, y ordena las filas por cantidad. Por eso representa un ranking de combinaciones libro/sede; un ranking total por libro requiere una agregación adicional explícita.

RP03 - Movimientos entre sedes

Selecciona remitos por fecha_despacho y agrupa por par origen/destino. Cuenta despachados y recibidos y calcula el promedio de (recepción - despacho) de los recibidos, en días. Los remitos en tránsito aportan a despachados pero no al denominador del promedio. Si no hay recepciones, muestra Sin recepciones, no un promedio artificial igual a cero.

En la implementación actual el rango selecciona la fecha de despacho; una recepción posterior al final del rango puede formar parte del promedio si el remito seleccionado ya tiene fecha_recepcion. Si se pide una foto histórica al final del período, habrá que acordar y agregar esa condición. No cambiar el significado sin ajustar título, filtros y pruebas.

RP04 - Disponibilidad por sede

Es un corte actual, no un filtro histórico. Agrupa ejemplares por libro y sede propietaria, cuenta total, disponibles y estados, y calcula disponibles/total × 100 redondeado para presentación. Copias prestadas, asignadas, en revisión o tránsito no son disponibles. Una sede de baja tampoco habilita sus copias por estar marcadas DISPONIBLE.

Conjunto inicial	Resultado esperado al corte 9/10/2026
Vencidos	Un préstamo vencido: PRE-0018, tres días de atraso.
Demanda, sin filtros de sede	Dos solicitudes interbibliotecarias válidas del 9/10: Rayuela/Norte y Ficciones/Central.
Movimientos	Un envío Central -> Norte despachado, aún sin recepción; sin promedio.
Disponibilidad	Seis copias disponibles sobre doce ejemplares del conjunto inicial.

Estos resultados pertenecen al seed del código y cambian cuando se realizan operaciones de demostración. Las cifras dibujadas en Pen son ilustrativas y no deben presentarse como resultados calculados del conjunto real.

Código de referencia: frontend/src/domain/reports.ts | frontend/src/domain/selectors.ts | frontend/src/pages/Reports.tsx | frontend/src/data/seed.ts

18 Qué backend existe y cómo ampliar su estructura

El backend tiene las apps common, sedes, socios, catalogo, circulacion, logistica y reportes registradas. En la copia revisada, sólo sedes tiene un modelo de dominio, migración inicial, serializer, selectores, servicios, vistas, rutas y pruebas completas de su módulo. No confundir una carpeta con una funcionalidad implementada.

App	Entidades y responsabilidad previstas
common	Errores uniformes, raíz de API y salud; comportamiento transversal.
sedes	Sede: administración, baja lógica y validación de habilitación del origen/destino.
socios	Socio: datos, duplicados, habilitar e inhabilitar.
catalogo	Libro, Autor, LibroAutor, Ejemplar: ficha, identidad física y consultas de disponibilidad.
circulacion	SolicitudPrestamo, Prestamo, Reserva: asignación, entrega, devolución y reservas; Strategy.
logistica	Remito, DetalleRemito, EventoRemito: trayecto, contenido, estados y trazabilidad; State.
reportes	Consultas agregadas verificables de las cuatro salidas.

Separación de responsabilidades dentro de una app

Archivo	Responsabilidad
models.py	Persistencia, relaciones y restricciones del esquema. Representa entidades y enumera estados.
services.py	Casos de uso que coordinan reglas, cambios relacionados y transacciones.
selectors.py	Consultas reutilizables sin mutaciones: listados, disponibilidad, detalle o agregaciones.
serializers.py	Entrada/salida HTTP, validación de formatos y campos controlados.
views.py	Permisos, validación del serializer, llamada al servicio y respuesta HTTP.
urls.py	Routers y rutas de operaciones. Coinciden con los contratos acordados.
admin.py	Administración consistente con los servicios; sin permitir saltos de estado.
migrations/	Historia versionada del esquema de base de datos.
tests/	Validación de modelo, servicio, API, patrones, cálculos y concurrencia cuando corresponda.

Se conserva el estilo de Sedes para los nuevos módulos. Las reglas no deben vivir sólo en el serializer o en botones de React. El servicio vuelve a comprobarlas con los registros actuales; la vista HTTP traduce el resultado al formato que la pantalla entiende.

Código de referencia: backend/apps/sedes/ | backend/config/settings.py | backend/README.md

19 De TypeScript al ORM sin cambiar el dominio

Representación del frontend	Representación prevista en Django	Observación
id: number	Clave primaria interna del ORM.	El cliente no asigna IDs reales.
texto: string	CharField o TextField según el atributo.	Definir longitudes sin alterar significado.
fecha: string	DateTimeField con zona horaria.	El serializer convierte ISO a fecha/hora y viceversa.
campo: string | null	Campo nullable cuando el modelo lo admite.	No volver obligatorio un dato que el DER permite ausente.
sede_origen_id: number	ForeignKey llamada sede_origen.	Django guarda un atributo interno sede_origen_id; el serializer debe mantener el nombre JSON acordado.
solicitud_id único en préstamo	Relación uno a uno o FK única.	Impide generar varios préstamos para una solicitud.
LibroAutor(libro_id, autor_id)	Relación muchos a muchos con asociación única.	Preservar el par definido en el DER.
Estado...	TextChoices más validación y restricción de valores.	choices no autoriza transiciones y no demuestra State.

FK no es autorización ni disponibilidad

Que un socio exista y se resuelva su FK no implica que esté habilitado para pedir material. Que una copia exista no implica que esté disponible. Las FK protegen referencias; los servicios protegen operaciones. El serializer puede usar una relación con queryset, pero el servicio revalida las condiciones que pueden haber cambiado.

Borrado y protección del historial

El dominio conserva préstamos, remitos y relaciones históricas. Para evitar que una baja administrativa destruya trazabilidad, se debe elegir una política de on_delete consistente, normalmente protección de registros referenciados para esas relaciones. Las bajas operativas siguen siendo estados; no introducir cascadas que borren préstamos al borrar una sede o socio.

Unicidad y comprobaciones estructurales

UniqueConstraint permite expresar unicidades y, cuando corresponde, limitar la protección a un conjunto de estados mediante una condición. CheckConstraint permite validar condiciones expresables sobre la fila, como fechas coherentes o tipo/origen/destino. Una restricción de una fila no puede comprobar por sí sola todas las condiciones en tablas relacionadas; esas reglas siguen en los servicios. [D2]

- Socio: número y documento únicos; Ejemplar: código único; Remito: número único.
- Libro: ISBN informado único; reservar ausencia como null.
- LibroAutor y DetalleRemito: no repetir los pares del DER.
- Solicitud: reserva vinculada única cuando no es null; coherencia de tipo con sedes.
- Prestamo: una solicitud por préstamo y fechas coherentes.
- Reserva: unicidad sólo de la combinación vigente, no de todo el historial.

Si se implementa LibroAutor con una tabla intermedia de Django, la elección de su clave técnica debe respetar el par único del DER. No agregar un nuevo identificador visible a React ni cambiar las cardinalidades por comodidad. Las migraciones son el lugar para documentar y verificar esa correspondencia.

Código de referencia: frontend/src/domain/models.ts | backend/apps/sedes/models.py | FigJam: DER y reglas de integridad

20 Contrato HTTP: lectura, creación y edición

LibraryApi es el contrato que usan las pantallas; http.ts es su traducción actual a HTTP. Las rutas siguientes, salvo Sedes y common, son propuestas contenidas en ese adaptador y todavía no están implementadas en Django. Los nombres se pueden confirmar durante el desarrollo, actualizando el adaptador sin cambiar los atributos del dominio.

Método	Ruta del patrón	Intención
GET	/api/{recurso}/	Consultar el listado, con paginación si se aplica.
GET	/api/{recurso}/{id}/	Consultar un registro. El snapshot actual usa los listados, pero la API de detalle permite evolución.
POST	/api/{recurso}/	Crear una entidad administrativa o iniciar un proceso permitido.
PATCH	/api/{recurso}/{id}/	Editar atributos autorizados; no transicionar estados libremente.
POST	/api/{recurso}/{id}/{accion}/	Ejecutar un caso de uso: entregar, devolver, recibir, habilitar, etc.

Recursos que consulta snapshot()

Array en Database	Ruta propuesta o existente	Permisos de escritura previstos
sedes	/api/sedes/	Sedes ya implementado.
socios	/api/socios/	Sólo operaciones autorizadas por su módulo.
libros	/api/libros/	Sólo operaciones autorizadas por su módulo.
autores	/api/autores/	Sólo operaciones autorizadas por su módulo.
ejemplares	/api/ejemplares/	Sólo operaciones autorizadas por su módulo.
solicitudes	/api/solicitudes/	Sólo operaciones autorizadas por su módulo.
prestamos	/api/prestamos/	Sólo operaciones autorizadas por su módulo.
reservas	/api/reservas/	Sólo operaciones autorizadas por su módulo.
remitos	/api/remitos/	Sólo operaciones autorizadas por su módulo.
detalle_remitos	/api/detalles-remito/	Lectura; historial no editable.
evento_remitos	/api/eventos-remito/	Lectura; historial no editable.

Además pide GET /api/libro-autores/. Ese método espera actualmente un array directo de pares libro_id y autor_id. Si se decide paginarlo con results, el adaptador debe cambiar: no puede asignar un objeto paginado a db.libro_autores.

Respuesta de listado compatible con DRF

```
{
  "count": 1,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 1,
      "numero_socio": "1042",
      "documento": "32456789",
      "nombre": "Lucía Fernández",
      "contacto": "lucia@example.com",
      "estado": "HABILITADO"
    }
  ]
}
```

list() admite un array directo o esta estructura con results y next. Recorre next hasta completar las páginas. La API debe conservar barras finales, nombres JSON y tipos de IDs; no devolver todos los vínculos anidados cuando el cliente espera números. Un serializer de salida puede incluir datos adicionales para otra vista, pero requiere una adaptación deliberada del consumidor.

Código de referencia: frontend/src/api/http.ts | frontend/src/api/contracts.ts | backend/config/settings.py

21 Catálogo completo de comandos HTTP

api.command(resource,id,action,input) usa POST. El servidor elige la transición, valida el estado previo y hace todos los cambios de la operación. La respuesta puede devolver el objeto actualizado con 200; el cliente actual ignora ese cuerpo y recarga snapshot(). Un 204 sin cuerpo también es tolerado por el lector, pero se debe elegir un contrato uniforme.

Recurso / acción	Payload que envía la interfaz	Efecto esperado
socios / habilitar o inhabilitar	{}	Cambiar habilitación con control del estado previo.
sedes / dar-de-baja o reactivar	{}	Operaciones existentes del módulo Sedes.
ejemplares / dar-de-baja	{}	Retirar una copia admisible sin borrar historial.
ejemplares / aprobar-disponibilidad	{}	Revisión apta y ubicación permitida para volver a disponible.
solicitudes / asignar	ejemplar_id	Asignar una copia compatible y no comprometida.
solicitudes / preparar-entrega	{}	Preparación de una solicitud local asignada.
solicitudes / entregar	fecha_entrega, fecha_vencimiento	Crear préstamo único y cambiar solicitud y copia.
solicitudes / cancelar	{}	Cancelar en estados autorizados y gestionar la copia.
prestamos / devolver	fecha_devolucion, condicion_devolucion	Cerrar el préstamo y llevar la copia a revisión.
reservas / asignar-reserva	ejemplar_id, fecha_limite_retiro	Preparar retiro y vincular su solicitud sin duplicación.
reservas / cancelar o expirar	{}	Cerrar reserva vigente y liberar lo que corresponda.
remitos / asignar	ejemplar_id	Agregar detalle durante preparación; asociar solicitud compatible.
remitos / empaquetar, despachar, registrar-salida, cancelar	responsable, observacion	Avanzar con State y agregar un evento de transición.
remitos / recibir	responsable, observacion, verificados: number[]	Verificar contenido, registrar recepción y actualizar relaciones.

Ejemplo de entrega

```
POST /api/solicitudes/43/entregar/
Content-Type: application/json

{
  "fecha_entrega": "2026-10-12T12:00:00-03:00",
  "fecha_vencimiento": "2026-10-26T12:00:00-03:00"
}
```

No se mandan estado=CUMPLIDA, socio duplicado, código nuevo de préstamo ni la sede propietaria copiada. La solicitud ya contiene sus relaciones. El servidor decide el resultado y crea el Préstamo. Antes de aceptar una fecha enviada, el serializer comprueba su formato y el servicio la compara con la entrega y las condiciones vigentes.

Ejemplo de recepción

```
POST /api/remitos/18/recibir/
{
  "responsable": "Personal de Sede Norte",
  "observacion": "Contenido cotejado con el remito.",
  "verificados": [5]
}
```

verificados contiene IDs de Ejemplar, no IDs de DetalleRemito. El mock los compara con todos los detalles del envío. En el backend debe definirse cómo el actor autenticado se representa en responsable y cómo se transporta la condición observada de cada detalle. El payload actual no cubre una inspección individual de condiciones o una recepción parcial.

Código de referencia: frontend/src/api/contracts.ts | frontend/src/api/http.ts | frontend/src/pages/Commands.tsx

22 Errores, contratos de campos y respuesta a conflictos

El backend existente usa un manejador que transforma los errores de DRF en error.status y error.detail. ApiError recoge mensaje, campos y código HTTP. Los formularios necesitan un diccionario donde las claves sean exactamente las que usan sus controles y los valores sean arrays de mensajes.

```
HTTP 400
{
  "error": {
    "status": 400,
    "detail": {
      "documento": ["Ya existe un socio con ese documento."]
    }
  }
}
```

Ese formato permite que Field muestre el mensaje debajo de Documento. Un error general puede usar detail: {"detail":"El ejemplar ya no está disponible."}. El cliente HTTP muestra ese texto como mensaje general. Si se devuelven cadenas sueltas por campo en vez de arrays, la pantalla actual no tiene el formato que espera y deberá normalizarlo.

Situación	Código a definir de forma consistente	Qué hará el frontend
Dato obligatorio, formato o duplicado	400, consistente con Sedes actual.	Conservar valores y mostrar errores por campo.
Actor no autorizado	401 o 403 según autenticación.	Informar y conducir al acceso o a la pantalla sin permiso; requiere implementar ese tratamiento.
Registro inexistente	404.	Mostrar ausencia y permitir volver al listado.
Estado cambió o copia ya comprometida	400 como la API existente, o 409 si se acuerda distinguir conflictos.	Refrescar y mostrar el motivo sin asumir que la operación sigue disponible.
Fallo de red	No hay respuesta HTTP; ApiError usa status=0.	Informar que no se pudo conectar y permitir reintentar de forma segura.
Error inesperado de servidor	500.	No tratarlo como confirmación ni como un nuevo estado del dominio.

Qué validar en cada capa

El campo HTML comprueba presencia y tipo básico para ayudar al operador. El serializer verifica el payload y las relaciones declaradas. El servicio verifica el proceso completo y los datos actuales. PostgreSQL protege el esquema y la unicidad. Estas verificaciones se complementan: una no cancela la necesidad de las otras. La vista HTTP debe mantener un formato estable de errores. [D7]

Reintentos y doble clic

busy evita repetir un clic en esa pantalla mientras espera. No evita que otra ventana o un reintento después de perder conexión repita el mismo POST. Entregar tiene protección por solicitud única; devolver verifica si ya se devolvió; una transición de remito valida el estado previo y evita repetir el evento. Si se necesita un mecanismo adicional de idempotencia, se define como contrato de la operación, sin inventar nuevos atributos de dominio por accidente.

Código de referencia: backend/apps/common/exceptions.py | frontend/src/api/contracts.ts | frontend/src/api/http.ts | frontend/src/components/ui.tsx

23 Implementar Socios: modelo y migración, paso a paso

> **Ejemplo didáctico, no código ya implementado**
>
> Los siguientes fragmentos muestran cómo construir el módulo con los atributos definidos. La copia revisada todavía no tiene models.py de Socios. Las longitudes de texto son decisiones técnicas iniciales coherentes con el formulario, no nuevos requisitos del diagrama.

1. Definir el modelo

```
# backend/apps/socios/models.py
from django.db import models

class EstadoSocio(models.TextChoices):
    HABILITADO = "HABILITADO", "Habilitado"
    INHABILITADO = "INHABILITADO", "Inhabilitado"

class Socio(models.Model):
    numero_socio = models.CharField(max_length=30, unique=True)
    documento = models.CharField(max_length=30, unique=True)
    nombre = models.CharField(max_length=150)
    contacto = models.CharField(max_length=255)
    estado = models.CharField(
        max_length=15,
        choices=EstadoSocio.choices,
        default=EstadoSocio.HABILITADO,
    )

    def __str__(self):
        return f"{self.numero_socio} - {self.nombre}"
```

Socio hereda de models.Model para que el ORM lo persista. Django agrega su clave primaria id. CharField conserva números documentales como texto sin eliminar ceros iniciales ni asumir que todos son enteros. unique=True protege la columna, mientras choices enumera sus opciones de presentación/validación. El estado inicial coincide con el diagrama.

Conviene agregar una restricción de valores válidos para estado en la base de datos y mantenerlo controlado por los servicios. El serializer no expondrá estado como un campo editable. Las reglas de quién habilita y por qué siguen pendientes; el ejemplo sólo representa el cambio manual acordado en la interfaz.

2. Generar y revisar la migración

```
REM Desde CMD, con el entorno de Python ya preparado
cd /d E:\universidad\DAO\tp-dao-G85\backend
.venv\Scripts\activate
python manage.py makemigrations socios
python manage.py migrate
python manage.py check
```

makemigrations genera la descripción del cambio del esquema; migrate lo aplica a la base configurada. Revisar el archivo antes de aplicar en un entorno compartido. La migración debe crear Socio con sus atributos y unicidades; no cargar credenciales ni datos personales reales. No eliminar migraciones ya aplicadas para ocultar

un cambio. [D6]

3. Verificación mínima de persistencia

Comprobar que un socio nuevo queda HABILITADO, que numero_socio y documento no admiten duplicados y que __str__ muestra una identificación útil. Confirmar que PostgreSQL conserva el registro después de reiniciar el backend. Los datos del navegador no migran automáticamente a la base: se crea un conjunto reproducible de demostración.

Código de referencia: frontend/src/domain/models.ts: Socio | frontend/src/pages/resources.ts: campos de socios | backend/apps/sedes/models.py como referencia de estilo

24 Implementar Socios: servicios, serializers y API

Servicios: operaciones que el sistema puede ejecutar

crear_socio recibe los cuatro atributos editables, limpia espacios periféricos y guarda el estado inicial. No decide que un documento con puntos equivale a otro sin puntos sin una política explícita. Habilitar e inhabilitar vuelven a leer y bloquear el Socio dentro de una transacción para comprobar su estado actual.

```
# Fragmento propuesto de services.py
from django.db import transaction
from .models import Socio, EstadoSocio

class HabilitacionSocioError(Exception):
    pass

@transaction.atomic
def cambiar_habilitacion(*, socio_id: int, nuevo_estado: str):
    socio = Socio.objects.select_for_update().get(pk=socio_id)
    if nuevo_estado not in EstadoSocio.values:
        raise HabilitacionSocioError("Estado no permitido.")
    if socio.estado == nuevo_estado:
        raise HabilitacionSocioError("El socio ya tiene ese estado.")
    socio.estado = nuevo_estado
    socio.save(update_fields=["estado"])
    return socio

def habilitar_socio(*, socio_id: int):
    return cambiar_habilitacion(
        socio_id=socio_id, nuevo_estado=EstadoSocio.HABILITADO
    )

def inhabilitar_socio(*, socio_id: int):
    return cambiar_habilitacion(
        socio_id=socio_id, nuevo_estado=EstadoSocio.INHABILITADO
    )
```

Estas funciones implementan las operaciones habilitar()/inhabilitar() del modelo a través de servicios de aplicación, como hace Sedes. La función interna no debe convertirse en un endpoint de PATCH libre de estado. Una inhabilitación no borra solicitudes y no cierra préstamos. La devolución no debe llamar a una validación de habilitación que impida devolver.

Serializer: el contrato de datos de la pantalla

```
# Fragmento propuesto de serializers.py
from rest_framework import serializers
from .models import Socio

class SocioSerializer(serializers.ModelSerializer):
    class Meta:
        model = Socio
        fields = [
            "id", "numero_socio", "documento", "nombre", "contacto", "estado"
        ]
        read_only_fields = ["id", "estado"]
```

ModelSerializer convierte el modelo a JSON y valida entradas según los campos. Complementar sus validadores para rechazar texto vacío después de strip() y conservar mensajes compatibles con el formulario. En la creación y edición, la restricción SQL sigue siendo necesaria para carreras; convertir sólo las violaciones de unicidad conocidas a mensajes de duplicado, sin ocultar cualquier IntegrityError como si fuera documento repetido. [D7]

Vista, selectores y router

SocioViewSet debe permitir listar, consultar, crear y editar los atributos personales. Las acciones detail=True de habilitar e inhabilitar llaman a servicios y traducen sus errores a ValidationError con el campo estado. El método get_queryset puede filtrar search y estado; debe rechazar estados desconocidos igual que Sedes.

```
# Rutas propuestas
GET   /api/socios/
GET   /api/socios/{id}/
POST  /api/socios/
PATCH /api/socios/{id}/
POST  /api/socios/{id}/habilitar/
POST  /api/socios/{id}/inhabilitar/
```

Registrar el router de la app y agregar path("api/socios/", include("apps.socios.urls")) a config/urls.py. Restringir DELETE y las escrituras de estado. Django Admin puede mostrar y buscar socios, pero debe llamar a los servicios para habilitación o mantener ese campo read-only. La estructura de ViewSet permite agrupar operaciones y conectar routers sin escribir una vista separada para cada URL. [D4]

Código de referencia: backend/apps/sedes/views.py | backend/apps/sedes/serializers.py | backend/apps/sedes/urls.py | frontend/src/api/contracts.ts

25 Catálogo y circulación: orden de construcción del backend

Catálogo primero: las operaciones necesitan copias reales

1. Crear Autor y Libro respetando atributos bibliográficos e ISBN opcional.

2. Representar LibroAutor con unicidad del par y salida compatible con la asociación del frontend.

3. Crear Ejemplar con código único, libro, propietaria, ubicación opcional, estado y condición física.

4. Implementar los serializers de lectura/escritura con las FK expuestas como IDs y los campos controlados read-only.

5. Implementar consultas de catálogo, copias por libro y disponibilidad; probar sedes de baja y material no apto.

6. Implementar actualización de autores sin duplicar asociaciones ni dejar operaciones parciales silenciosas.

Circulación local después

1. Crear SolicitudPrestamo y Prestamo conservando la solicitud única del préstamo y las relaciones definidas.

2. Implementar crear solicitud y asignar copia con control de concurrencia sobre la misma copia.

3. Implementar preparar entrega local y entrega; registrar fechas, pedido cumplido y copia prestada juntos.

4. Implementar devolución, condición física y estado de revisión; permitirla aunque el socio esté inhabilitado.

5. Implementar aprobación de disponibilidad según condición y política de ubicación definida.

6. Exponer acciones HTTP y probar el recorrido desde React con información real.

Reservas y logística completan el recorrido

Reserva depende de socios, libros y sedes, y su asignación debe coordinarse con solicitudes y copias. Logística depende de copias y solicitudes para relacionar detalles; no puede producir transporte real solamente cambiando el color de una etiqueta. Crear sus entidades y evento inicial, luego implementar State, despacho, recepción y la coordinación de circulación.

Las dependencias no exigen construir todo el backend antes de validar un módulo. Es conveniente conectar Socios y Sedes primero con una carga de datos adaptada para esa etapa, y después demostrar el préstamo local con PostgreSQL antes de sumar traslado y reservas. Cada etapa produce una capacidad observable y probada.

Préstamo: sólo se crea por entrega

El endpoint genérico POST /api/prestamos/ no debería aceptar cualquier solicitud y fechas para saltar preparación o recepción. El cliente ya crea el préstamo mediante /api/solicitudes/{id}/entregar/. La lectura de Prestamo es independiente; su creación pertenece a ese caso de uso.

Código de referencia: frontend/src/domain/models.ts | frontend/src/api/mock.ts | docs/plan-proyecto.md

26 Transacciones y concurrencia: la parte que el mock no garantiza

Una operación de circulación puede modificar más de una fila. Si se crea el préstamo pero no se marca la copia PRESTADO, la disponibilidad queda incoherente. Si se actualiza el estado del remito pero no se escribe el evento, el historial pierde trazabilidad. Cada conjunto debe confirmarse completo o no confirmarse.

transaction.atomic agrupa una unidad de trabajo: una salida correcta confirma sus cambios y una excepción permite deshacerlos. Evitar capturar IntegrityError dentro de un bloque y seguir consultando como si la transacción no estuviera afectada; capturarlo fuera del bloque apropiado o usar un savepoint interno y tratar el error después del rollback. [D1]

select_for_update bloquea las filas seleccionadas durante la transacción en bases compatibles como PostgreSQL. No basta con usar atomic si dos operaciones pueden leer la misma copia disponible antes de que alguna la cambie. La prueba de concurrencia debe ejecutarse sobre PostgreSQL con conexiones distintas y transacciones reales. [D5]

Esqueleto conceptual de asignación

```
# Pseudocódigo: los nombres se ajustan a los modelos de Django
with transaction.atomic():
    # Mantener el mismo orden de bloqueos en todos los servicios.
    solicitud = bloquear_solicitud(solicitud_id)
    ejemplar = bloquear_ejemplar(ejemplar_id)
    socio = bloquear_y_validar_socio(solicitud.socio_id)
    validar_sedes_y_estado_actual(solicitud)
    validar_libro_origen_y_disponibilidad(solicitud, ejemplar)

    solicitud.asignarEjemplar(ejemplar)
    ejemplar.asignar()
    guardar_solicitud_y_ejemplar()
# Al salir, la operación está confirmada o se deshizo completa.
```

Este ejemplo explica la unidad de trabajo, no es un servicio listo para copiar. Las funciones bloquear... y los métodos finales se diseñarán en el módulo; la verificación debe mantener todas las invariantes del modelo y el orden de bloqueo acordado.

Bloquear también aquello que puede invalidar una decisión

Si la entrega requiere un socio habilitado y sedes activas, se debe coordinar esa comprobación con las operaciones que inhabilitan o dan de baja. Bloquear sólo el ejemplar no impide por sí mismo que la habilitación cambie al mismo tiempo. Definir un protocolo de bloqueos consistente entre los servicios evita aceptar operaciones con premisas que dejaron de cumplirse y reduce el riesgo de bloqueos cruzados.

Restricciones complementarias

La unicidad de Prestamo.solicitud protege una segunda entrega. La unicidad condicionada de reserva protege duplicados vigentes. La unicidad de DetalleRemito(remito, ejemplar) protege repetición dentro del mismo envío. La exclusión de una copia entre distintas operaciones vigentes puede requerir servicios coordinados y bloqueos, porque atraviesa más de una tabla. No agregar un campo de estado redundante en Prestamo sólo para facilitar una restricción.

Las operaciones de reintento deben volver a comprobar el estado. Una segunda recepción no crea un evento extra ni mueve de nuevo una copia ya recibida; una segunda devolución no cambia la primera fecha. Las respuestas deben informar el resultado vigente y evitar que el operador repita una escritura cuyo primer intento tal vez sí llegó al servidor.

Código de referencia: backend/apps/sedes/services.py | frontend/src/api/mock.ts | frontend/src/domain/workflows.test.ts

27 State para Remito: clases con comportamiento

El requisito es implementar State como patrón, no solamente un campo enumerado. Remito puede persistir su código de estado, pero las operaciones permitidas y el comportamiento de transición se distribuyen en clases que representan cada estado. El historial continúa siendo EventoRemito por separado.

Clase de estado propuesta	Operaciones que autoriza	Controles relevantes
EnPreparacion	Agregar contenido, empaquetar, cancelar.	Contenido válido; empaquetar exige al menos una copia.
Empaquetado	Despachar o cancelar.	Origen/destino activos, contenido consistente, socios habilitados para solicitudes del envío.
Despachado	Registrar salida.	No cancelar por la vía previa al despacho.
EnTransito	Recibir.	Contenido verificado y reglas de recepción acordadas.
Recibido	Consulta; no repetir transición.	Fechas y evento final ya registrados.
Cancelado	Consulta; no revivir sin una regla explícita.	Historial conservado y asignaciones gestionadas.

```
# Esquema didáctico del patrón; no implementación completa
class EstadoRemitoBase:
    def despachar(self, contexto):
        raise TransicionNoPermitida("No se puede despachar desde este estado.")

class Empaquetado(EstadoRemitoBase):
    def despachar(self, contexto):
        contexto.validar_sedes_activas()
        contexto.validar_contenido_y_socios()
        contexto.registrar_despacho()
        contexto.actualizar_copias_y_solicitudes()
        contexto.transicionar_a(Despachado())

# Servicio: abre la transacción, bloquea Remito y sus recursos,
# obtiene el objeto Estado y llama a su operación.
# La transición registra estado anterior/nuevo y EventoRemito.
```

El contexto reúne el remito y colaboradores que realizan cambios coordinados. Una fábrica o registro puede convertir el código persistido en un objeto de estado; esa selección no reemplaza la conducta de las clases. El

servicio controla la transacción y la autorización. El objeto State decide si la operación es válida desde ese estado y ejecuta la conducta específica.

Cómo demostrar el patrón en la entrega del TP

- Mostrar las clases de estados y el contrato común de operaciones.
- Ejecutar despacho desde Empaquetado y comprobar fecha, evento, copia y solicitud.
- Intentar despacho desde EnPreparacion y comprobar rechazo sin cambios parciales.
- Intentar cancelar desde Despachado o EnTransito y comprobar que la clase no lo autoriza.
- Recorrer el historial con responsable, instantes y estados anteriores/nuevos.

El mock usa un mapa de transiciones y ramas condicionales. Es material de aceptación del frontend, no evidencia de que el patrón State ya esté implementado. La implementación real debe respetar esos estados del modelo y los límites confirmados, ampliando sólo las reglas que el equipo resuelva.

Código de referencia: docs/definicion-producto.md: State | frontend/src/api/mock.ts: rama remitos | FigJam: máquina de estados de Remito

28 Strategy para la tramitación local e interbibliotecaria

Strategy representa dos modos intercambiables de tramitar una solicitud bajo un contrato común. La solicitud conserva su tipo y sus relaciones. La estrategia local prepara material en la misma sede; la interbibliotecaria coordina el traslado necesario antes de la entrega. Las validaciones comunes se reutilizan en lugar de copiarse en ambas clases.

```
# Esquema conceptual, no código ya instalado
class TramitacionPrestamo:
    def tramitar(self, contexto):
        raise NotImplementedError

class TramitacionLocal(TramitacionPrestamo):
    def tramitar(self, contexto):
        contexto.validar_origen_igual_destino()
        contexto.asignar_ejemplar()
        contexto.preparar_entrega_local()

class TramitacionInterbibliotecaria(TramitacionPrestamo):
    def tramitar(self, contexto):
        contexto.validar_origen_distinto_destino()
        contexto.asignar_ejemplar()
        contexto.dejar_pendiente_el_circuito_de_remito()

# Las sedes, socio, libro y copia se validan en el caso de uso.
# El cliente no decide una clase Python ni salta la recepción.
```

El contexto expone las operaciones necesarias y el servicio selecciona la estrategia coherente con origen/destino o valida el tipo recibido. No hay obligación de crear un remito nuevo por cada solicitud: el modelo permite agrupar detalles. La estrategia interbibliotecaria no debe duplicar préstamos ni convertir la fecha de despacho en fecha de entrega.

State y Strategy se complementan

Strategy responde cómo se tramita el pedido según el circuito. State responde qué puede hacer ahora el remito según su situación. El servicio coordina ambas áreas con sus entidades. No se necesita introducir clases de estado para Libro, cuya máquina inicial no está prevista, ni llamar State a cualquier TextChoices.

Criterios de validación

- Las dos estrategias implementan el mismo contrato y pueden seleccionarse sin cambiar el consumidor.
- La local rechaza origen distinto de destino; la interbibliotecaria rechaza origen igual a destino.
- Una solicitud interbibliotecaria no puede entregarse antes de la recepción y preparación.
- Las reglas comunes de socio, sedes, identidad de copia y exclusión de asignaciones siguen vigentes en ambas.
- Las pruebas muestran comportamientos distintos, no sólo el nombre de dos clases vacías.

Código de referencia: docs/definicion-producto.md: Strategy | frontend/src/domain/models.ts: tipo de solicitud | FigJam: solicitudes locales e interbibliotecarias

29 Autenticación, permisos y separación entre Socio y usuario

El acceso de demostración compara biblioteca/demo y navega a /inicio. No crea una sesión del servidor, no guarda un token y no protege las rutas. Si el adaptador es HTTP, ese formulario informa que falta implementar el acceso real. La pantalla sin-acceso es visual; no es una regla de autorización.

La configuración actual del backend establece AllowAny como permiso por defecto. Django tiene su sistema de usuarios, pero eso no implica que cada endpoint del proyecto esté protegido. Autenticación identifica al actor; permisos deciden qué puede hacer. IsAuthenticated protege una operación de un actor anónimo, pero la autorización por sede o por operación requiere criterios adicionales acordados. [D3, D8]

Socio no es cuenta de personal

Socio representa a quien solicita el libro. El usuario autenticado representa al empleado que registra una entrega, administra habilitación o recibe un envío. No agregar password, email de acceso o una FK a User dentro de Socio sólo para implementar el formulario de acceso. La definición de producto indica que un socio no necesita automáticamente una cuenta.

Contrato de acceso propuesto para cookies de sesión

Endpoint propuesto	Intención	Qué falta construir
GET /api/auth/csrf/	Preparar la protección CSRF.	Vista que emita la cookie o token según configuración.
POST /api/auth/login/	Validar credenciales y crear sesión.	Vista protegida correctamente contra CSRF y errores de acceso.
GET /api/auth/me/	Consultar el actor y la sesión vigente.	Datos mínimos para el contexto de cuenta; permisos acordados.
POST /api/auth/logout/	Cerrar la sesión del servidor.	Invalidar sesión y limpiar estado del cliente.

Estas rutas no están implementadas. Se proponen sesiones porque http.ts ya envía credentials="include" y toma csrftoken para X-CSRFToken. Se puede elegir otra autenticación, pero exigiría ajustar el adaptador y las pruebas; no se presenta JWT como una obligación nueva del proyecto.

Cambios concretos en React

1. Crear un contrato de autenticación y un cliente de acceso; no mezclar una cuenta del personal con las entidades de Database.

2. Adaptar Access para llamar login y consultar la sesión real, conservando errores del formulario.

3. Agregar estado de actor y carga inicial de sesión a un proveedor o hook apropiado.

4. Proteger la navegación de las rutas privadas y gestionar respuestas 401/403.

5. Hacer que cerrar sesión invoque logout y limpie el contexto; el enlace actual sólo navega al acceso.

6. Mostrar responsable de operaciones a partir del actor autorizado. Si el modelo conserva responsable como texto, registrar su representación autorizada sin inventar atributos nuevos.

CSRF y CORS no son lo mismo

CSRF protege peticiones de modificación con cookies. La inicialización de la cookie, el envío del token y los orígenes confiables deben coincidir con el despliegue real; login no debe quedar exento sólo porque una sesión aún no exista. El código actual envía el token si encuentra su cookie, pero no implementa su obtención inicial. [D9]

CORS regula lectura desde otro origen y no reemplaza autenticación. El proxy /api mantiene el mismo origen para el navegador y simplifica la comunicación. La configuración actual de CORS permite localhost:5173, mientras la vista previa usa 127.0.0.1:5173. Si se llama directamente a otro origen, esas direcciones deben revisarse; para CSRF puede hacer falta agregar un ajuste de orígenes confiables, hoy no presente en settings.py.

Código de referencia: frontend/src/pages/Other.tsx: Access | frontend/src/api/http.ts | backend/config/settings.py

30 Fechas, zonas horarias y vencimientos

El transporte utiliza fechas ISO 8601. Un instante como 2026-10-12T12:00:00-03:00 indica mediodía en Buenos Aires. El mismo instante puede expresarse como 2026-10-12T15:00:00Z. No son dos entregas diferentes: cambia la representación de zona horaria.

El backend tiene USE_TZ=True y TIME_ZONE=America/Argentina/Buenos_Aires. Para implementar servicios debe usar fechas conscientes de zona y conservar un criterio uniforme al serializar y comparar. timezone.now() evita depender de la hora local del servidor. Los cortes por fecha se interpretan según la zona acordada, no según el navegador de otro país. [D10]

Quién establece cada fecha

Fecha	Responsabilidad prevista
fecha_solicitud / fecha_reserva	Servidor al registrar la operación.
fecha_creacion de Remito	Servidor junto con su evento inicial.
fecha_hora de EventoRemito	Servidor al confirmar la transición.
fecha_despacho / fecha_recepcion	Servidor al registrar esas operaciones, salvo política explícita de fechas operativas retroactivas.
fecha_entrega / fecha_devolucion	Formulario actual envía fecha/hora; serializer y servicio validan según la política acordada.
fecha_vencimiento	Debe ser posterior a la entrega; el plazo se define explícitamente.
fecha_limite_retiro	Se ingresa al preparar una reserva; duración automática todavía pendiente.

Vencido y días de atraso

isOverdue() compara el instante actual con fecha_vencimiento y exige que no haya devolución. daysLate() calcula días de calendario para presentación. Por eso un préstamo vencido dentro de hoy puede estar vencido con cero días completos de diferencia entre fechas. En un reporte histórico se evalúa si estaba abierto al corte, aunque hoy ya se haya devuelto.

Para valores datetime-local, CommandDialog agrega el offset -03:00 al enviar. La sugerencia de las 12:00 no es una política temporal del dominio. Cuando el equipo defina si el operador puede registrar hechos retroactivos o futuros, esa regla debe comprobarse en backend y comunicarse en el formulario, sin inferirla de un valor por defecto.

Errores típicos que hay que evitar

- Comparar cadenas con formatos distintos en lugar de instantes o fechas normalizados.
- Perder el offset al pasar de input HTML a JSON.

- Usar fecha_despacho como inicio del plazo del préstamo.
- Persistir un estado VENCIDO y además calcularlo, generando dos fuentes que pueden discrepar.
- Contar el vencimiento histórico usando sólo el estado actual de devolución.
- Cambiar el criterio de días de atraso sin actualizar pruebas y explicación del reporte.

Código de referencia: frontend/src/domain/selectors.ts | frontend/src/pages/Commands.tsx | frontend/src/domain/reports.ts | backend/config/settings.py

31 Integrar el backend de manera progresiva

El cambio de mock a HTTP está preparado como selección de adaptador, pero el backend incompleto no puede satisfacer todavía el snapshot entero. Prometer que basta cambiar VITE_DATA_SOURCE a http sería incorrecto: hoy se pedirían socios, libros, autores, copias, circulación y logística, y el fallo de cualquiera rechaza la carga global.

Opción para la primera integración

Como propuesta técnica, incorporar lecturas por módulo con un adaptador o servicio específico de Socios. El módulo puede consultar listados y detalles reales sin necesitar catálogos o remitos todavía no implementados. Si su ficha muestra solicitudes, esa sección debe indicar que aún no está integrada o cargarlas cuando Circulación exista, en lugar de mezclar IDs de mock con registros reales.

Otra opción es esperar a que todos los recursos de lectura estén disponibles antes de cambiar el adaptador global. Es más simple para el snapshot actual pero retrasa la verificación integrada. La recomendación para el equipo es integrar por capacidad y adaptar la lectura del frontend; no crear endpoints vacíos que aparenten procesos terminados.

Secuencia concreta de conexión de Socios

1. Implementar modelo, migración, serializer, servicios y rutas de Socios; verificar el JSON en Swagger.

2. Crear un pequeño conjunto de datos reproducible en PostgreSQL. No trasladar el localStorage como si fuera información autorizada.

3. Adaptar el proveedor o lector de Socios para obtener sus entidades desde Django y sus relaciones sólo cuando estén disponibles.

4. Probar listado, búsqueda, alta y edición con IDs y campos reales.

5. Probar habilitar/inhabilitar y errores de duplicado; comprobar que el estado no se modifica por PATCH directo.

6. Actualizar la recarga y el manejo de errores para que un fallo en otro módulo no bloquee Socios.

7. Proteger las rutas y operaciones con el acceso real cuando su contrato esté disponible.

No mezclar dos fuentes de verdad en el mismo proceso

Una solicitud real no debe vincular un socio que sólo existe en el seed del navegador. Los IDs del mock no están reservados en PostgreSQL; pueden corresponder a otras personas o libros. Una migración gradual separa claramente datos reales y demostración, y cada proceso integrado usa relaciones del mismo origen.

Adaptaciones necesarias del lector actual

- Agregar validación de los JSON recibidos; los casts as Database no comprueban tipos en ejecución.
- Gestionar estados de carga y error por módulo para poder integrar parcialmente.
- Definir cómo las páginas obtienen las entidades relacionadas sin asumir siempre un snapshot completo.
- Resolver los errores de recarga después de una escritura confirmada.
- Pasar filtros y paginación al servidor cuando el volumen lo requiera.
- Conservar nulabilidad y nombres de las FK, y adaptar explícitamente serializers que los representen de otro modo.

Código de referencia: frontend/src/api/index.ts | frontend/src/api/http.ts: snapshot | frontend/src/app/context.tsx | frontend/src/pages/Details.tsx

32 Implementar los reportes en Django y conectar sus salidas

La primera demostración calcula reportes sobre Database. El backend deberá realizar consultas consistentes sobre PostgreSQL, con filtros y resultados verificables. Crear endpoints de reporte no modifica automáticamente ReportPage: todavía invoca buildReport() en el cliente y habría que agregar un consumidor de esas respuestas.

Endpoint propuesto	Filtros	Definición a conservar
GET /api/reportes/vencidos/	fecha_corte, sede_entrega_id	Préstamos abiertos en el corte; días de atraso y agrupación por entrega.
GET /api/reportes/demanda/	desde, hasta, sede_solicitante_id	Solicitudes interbibliotecarias válidas por fecha de solicitud.
GET /api/reportes/movimientos/	desde, hasta, origen/destino o sede	Envíos por despacho; promedio sólo entre recibidos.
GET /api/reportes/disponibilidad/	sede_propietaria_id, libro_id si se acuerda	Corte actual; totales, estados y proporción disponible.

Estos endpoints son una propuesta adicional: no aparecen como implementados ni se llaman todavía desde http.ts. Los nombres finales de filtros, el shape de respuesta y la agrupación deben acordarse entre frontend y backend antes de sustituir el cálculo local.

Ejemplo de estructura de resultado propuesta

```
{
  "fecha_corte": "2026-10-09",
  "zona_horaria": "America/Argentina/Buenos_Aires",
  "cantidad": 1,
  "resultados": [
    {
      "prestamo_id": 18,
      "sede_entrega_id": 1,
      "dias_atraso": 3
    }
  ]
}
```

Es una proyección de reporte, no una nueva entidad del dominio. Puede incluir etiquetas o nombres para presentar, sin almacenarlos duplicados dentro de Prestamo. La estructura de ReportResult actual usa headers, rows, summary y rule; se puede crear un adaptador que traduzca una respuesta tipada del backend a esa representación visual.

Consultas y validación independiente

Usar el ORM para filtrar y agrupar evitando multiplicar filas por un join con autores o detalles. En demanda, contar solicitudes; en movimientos, contar remitos; en disponibilidad, contar ejemplares. Un COUNT sobre una unión incorrecta puede duplicar resultados aunque la pantalla se vea bien.

Calcular resultados esperados con un conjunto pequeño preparado manualmente, luego contrastar cada salida. Incluir cortes históricos, períodos vacíos, envíos sin recepción y sedes de baja. La disponibilidad es un corte actual; obtener su historia exigiría datos suficientes del historial y una decisión de alcance, no cambiar el nombre de un filtro.

Código de referencia: frontend/src/domain/reports.ts | frontend/src/pages/Reports.tsx | docs/definicion-producto.md: RP01-RP04

33 Desarrollo local, variables de entorno y comandos

Frontend en CMD

```
cd /d E:\universidad\DAO\tp-dao-G85\frontend
npm ci
npm run dev

REM Verificar el código y la compilación
npm test
npm run build

REM Ver la compilación local (no es el servidor Django)
npm run preview
```

npm ci instala las versiones del lockfile. npm run dev utiliza el puerto 5173 y falla si ya está ocupado, porque strictPort está activado. npm run preview sirve dist para revisar la salida compilada. Si el servidor ya está abierto, reutilizarlo o cerrarlo antes de iniciar otro en el mismo puerto.

Variables de Vite

```
# frontend/.env (archivo local, creado desde .env.example)
VITE_DATA_SOURCE=mock
VITE_API_BASE_URL=/api

# Cuando existan los endpoints y la integración esté adaptada:
VITE_DATA_SOURCE=http
VITE_API_BASE_URL=/api
```

Sólo el valor exacto http selecciona el adaptador real; cualquier otro valor mantiene el mock en el código actual. Reiniciar Vite después de cambiar la configuración. Las variables VITE_ quedan en el bundle del cliente: no contienen contraseñas de PostgreSQL ni secretos de Django.

Backend local en CMD

```
cd /d E:\universidad\DAO\tp-dao-G85\backend
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements\development.txt
python manage.py migrate
python manage.py runserver 127.0.0.1:8000
```

Estos comandos suponen PostgreSQL accesible y variables correctas. settings.py usa os.getenv, pero no carga automáticamente un .env local mediante una biblioteca. Copiar .env.example en la raíz es suficiente para el env_file de Compose, no para exportar todas sus variables al proceso Python ejecutado directamente. En CMD se pueden establecer con set NOMBRE=valor antes de runserver, o usar el mecanismo de carga que el equipo acuerde.

Proxy de desarrollo

Vite reenvía /api a http://127.0.0.1:8000. El navegador puede pedir /api/socios/ al mismo origen donde ve React. Evitar cambiar arbitrariamente esa base a una URL de otro host sin revisar cookies, CORS, CSRF y el manejo de enlaces de paginación del adaptador.

Código de referencia: frontend/package.json | frontend/.env.example | frontend/vite.config.ts | backend/config/settings.py | backend/README.md

34 Docker y despliegue reproducible

El Dockerfile del frontend compila con Node 24 y publica dist con Nginx. Las dependencias se instalan con npm ci. La imagen final no necesita el servidor de desarrollo de Vite: sirve los archivos ya compilados y conserva las rutas de la SPA con try_files.

Dos rutas de entrada

Ruta	Destino en Nginx	Razón
/catalogo/2, /socios/1, etc.	index.html cuando no existe un archivo.	React Router puede abrir rutas directas y recargarlas.
/assets/... y /brand/...	Archivos estáticos del build.	CSS, JS, imágenes y logos se sirven desde dist.
/api/...	Proxy al servicio backend:8000.	Conserva una base /api común para la interfaz.

Nginx usa el nombre backend de la red Docker, no 127.0.0.1 dentro del contenedor. Su resolución se realiza con el DNS interno. Si Django no está disponible, la interfaz de mock puede seguir sirviéndose, pero las peticiones reales de API fallarán; eso no demuestra que el backend esté conectado.

Configuración de Compose

frontend expone el puerto 5173 del host hacia el 80 de su contenedor. database usa PostgreSQL 17 y un volumen persistente. backend aplica migraciones y arranca Gunicorn; espera el healthcheck de PostgreSQL. El frontend no tiene una dependencia obligatoria del backend, porque la demostración puede funcionar sin él.

```
REM Desde la raíz del proyecto, en CMD
cd /d E:\universidad\DAO\tp-dao-G85

REM Sólo si todavía no existe el archivo local de configuración
if not exist .env copy .env.example .env

REM Revisar las variables antes de iniciar los servicios
docker compose config
docker compose up --build -d
docker compose logs backend

REM Ejecutar una comprobación dentro del contenedor
docker compose exec backend python manage.py check
```

Editar .env con los valores del entorno de trabajo antes de usar el Compose. No sobrescribir un archivo real existente con el ejemplo. El documento no reproduce secretos. La comprobación conjunta requiere ese archivo: durante la revisión previa faltaba en la copia del proyecto.

Los valores del frontend se fijan al compilar

VITE_DATA_SOURCE y VITE_API_BASE_URL son argumentos del build. Cambiar la variable después de construir la imagen no transforma el JS ya generado. Para pasar a http, configurar el valor y reconstruir el frontend. Los modelos y las pantallas no cambian por ese paso, pero todos los contratos consumidos sí deben estar implementados.

Criterios de despliegue terminado

- Clon limpio: instalar o construir, migrar y cargar datos de demostración siguiendo comandos verificados.
- Abrir una ruta profunda de React y recargarla sin 404 de Nginx.
- Conectar API real y comprobar cookies/permisos, no sólo la pantalla de catálogo.

- Crear un registro, reiniciar los servicios y comprobar persistencia en PostgreSQL.
- Separar configuración de desarrollo de la desplegada; fijar DEBUG, hosts y secretos según el entorno.
- Verificar la disponibilidad real de base y API; el endpoint health fijo no basta para certificar la conexión.

Código de referencia: frontend/Dockerfile | frontend/nginx.conf | docker-compose.yml | backend/apps/common/views.py

35 Plan de pruebas: desde el modelo hasta el navegador

Las 13 pruebas actuales pertenecen al frontend y ejercitan funciones del mock y los reportes. No realizan peticiones a Django ni validan transacciones reales de PostgreSQL. Sus casos de comportamiento sirven como referencia para escribir pruebas de backend, pero se deben comprobar resultados esperados sin depender únicamente del código que se está verificando.

Prueba actual	Qué cubre
1	completa préstamo local y devolución sin duplicar socio/libro en Prestamo
2	inhabilitar impide nuevos préstamos y no impide devoluciones
3	rechaza documentos y números de socio duplicados
4	impide doble asignación del mismo ejemplar
5	recorre remito, preserva propietario e historial y habilita entrega después de recibir
6	reserva requiere plazo explícito y se cumple con la entrega
7	no permite iniciar operaciones con sedes dadas de baja
8	una operación inválida no se persiste
9	calcula tres días de atraso en el corte de ejemplo
10	conserva el vencido histórico aunque se devuelva después del corte
11	excluye canceladas de la demanda y usa destino como solicitante
12	no calcula promedio de remitos aún no recibidos
13	calcula disponibilidad sobre todos los ejemplares y admite filtros vacíos

Capas de pruebas propuestas para Django

Capa	Ejemplos de verificación
Modelo / restricciones	Unicidad de socio, copia y remito; relación única solicitud/préstamo; null admitidos; FK e integridad de fechas.
Servicios de administración	Alta con duplicado, edición válida, cambio de habilitación repetido y conservación del historial.
Servicios de circulación	Asignación correcta y rechazos, entrega única, vencimiento, devolución por socio inhabilitado y material no apto.
State / Strategy	Operaciones permitidas y rechazadas según el estado; diferencias verificables entre circuitos.
API	Payloads, nombres de campos, códigos HTTP, errores por campo, filtros y paginación.
Concurrencia en PostgreSQL	Dos conexiones asignan la misma copia; dos entregas del mismo pedido; reserva contra solicitud por la misma copia.
Reportes	Conteos conocidos, corte histórico, denominador de promedio, joins sin duplicación y datos vacíos.
Integración en navegador	Recorridos con API y BD reales, sesión, rechazo visible, recarga y persistencia tras reinicio.

Comandos existentes del backend

```
REM Con el entorno de Python activado
python manage.py check
python manage.py makemigrations --check --dry-run
python -m pytest
ruff check .
ruff format --check .
```

SQLite es una facilidad configurada para pruebas aisladas, no una prueba equivalente de bloqueos concurrentes de PostgreSQL. Una comprobación de makemigrations detecta diferencias de esquema pendientes; no prueba por sí misma las reglas del dominio. Una prueba de API no reemplaza una prueba de una carrera real entre conexiones.

Regresiones que deben agregarse al integrar

- Fechas con offset y vencimiento dentro del día; distinguir instante vencido y días de atraso.
- EventoRemito creado junto con la transición, y ningún evento nuevo si la operación falla.
- Autor/LibroAutor actualizados sin quedar un libro parcialmente guardado.
- Recepción con condición física observada, una vez definida su política.
- Operaciones desde/hacia sedes de baja, incluyendo cambios de habilitación concurrentes.
- Sesión vencida durante una escritura y fallo de recarga después de una operación confirmada.

Código de referencia: frontend/src/domain/workflows.test.ts | backend/apps/sedes/tests/ | backend/README.md

36 Hoja de ruta para implementar sin perder el control

El orden siguiente propone unidades de trabajo verificables. No redefine el alcance ni fija horas inventadas. Los responsables y fechas del plan de sprints se conservan como planificación del equipo. Cada etapa se cierra con una capacidad integrada y un conjunto de rechazos observables.

Etapa	Trabajo	Evidencia para pasar a la siguiente
1	Revisar contratos, decisiones abiertas y entorno.	Modelos coincidentes y PostgreSQL/API ejecutables.
2	Implementar Socios siguiendo Sedes.	Alta, edición, unicidad y habilitación verificadas.
3	Preparar carga gradual e integrar Socios y Sedes.	Las pantallas operan con información real sin depender de endpoints inexistentes.
4	Implementar Libro, Autor, LibroAutor y Ejemplar.	Identidad, propietaria, ubicación, disponibilidad y migraciones coherentes.
5	Implementar circulación local y Strategy local.	Solicitar, asignar, entregar, vencer y devolver con reglas y concurrencia.
6	Implementar logística y State.	Remito desde preparación a recepción, historial persistido y rechazos de transición.
7	Completar Strategy interbibliotecaria.	Traslado, recepción y entrega al socio sin cambiar propiedad ni plazo por transporte.
8	Implementar reservas con reglas confirmadas.	Espera, asignación, retiro, cancelación y expiración coherentes.
9	Implementar autenticación y permisos acordados.	Actor real y operaciones restringidas; no sólo pantalla de acceso.
10	Implementar e integrar cuatro reportes.	Resultados contrastados con datos de referencia independientes.
11	Cerrar Docker y pruebas integradas.	Clon limpio, rutas profundas, persistencia, sesión y circuitos principales.
12	Actualizar modelos, documentación y demostración.	Equipo puede explicar entidades, patrones, contratos y decisiones.

Proceso para cada tarea

1. Elegir una operación concreta y el requisito que satisface.

2. Identificar entidades, atributos, estados previos y resultado esperado.

3. Acordar payload, salida, errores y permisos antes de escribir ambas puntas.

4. Escribir modelo/migración si falta persistencia; después servicio y consulta.

5. Agregar serializer, vista y ruta sin permitir atajos de estado.

6. Probar éxito, rechazo y riesgo relevante como carrera o cálculo.

7. Conectar la pantalla y verificar el recorrido real.

8. Pedir revisión humana y registrar evidencia de aceptación.

Código de referencia: docs/plan-proyecto.md | docs/definicion-producto.md

37 Decisiones abiertas y brechas de integración

Los modelos existentes se mantienen como referencia vigente. Algunas reglas se declaran pendientes en esos mismos modelos/documentos. Resolverlas no autoriza a rediseñar por cuenta propia las entidades: se acuerda el comportamiento, se verifica su compatibilidad y sólo se modifica un modelo si el equipo lo decide explícitamente.

Tema	Estado actual	Resultado que se necesita
Lugar de devolución y retorno	Pendiente de definición; mock conserva ubicación y manda a revisión.	Dónde se recibe, cuándo puede volver a disponible y cómo registrar retorno a propietaria.
Reservas y prioridad	Sin FIFO ni selección automática de la siguiente reserva.	Criterio de orden, elegibilidad y exclusión concurrente.
Plazo de retiro	Ingreso explícito; no duración automática.	Quién lo fija y cómo se controla la expiración.
Recepción parcial / incidencias	Mock exige todo y copia la condición conocida.	Recepción admitida y condición observada por detalle; tratamiento de material no apto.
Sedes dadas de baja	Baja actual sin comprobación de procesos activos.	Qué se puede completar y qué se bloquea entre procesos ya iniciados.
Catálogo sin ISBN	Atributo opcional; sin política confirmada de deduplicación.	Criterio por edición u otros datos suficientes.
Habilitación y permisos	Operación manual simulada; AllowAny en API actual.	Actor autorizado, razones y alcance de las operaciones.
Carga HTTP parcial	snapshot depende de todos los recursos.	Lectura por módulo o backend suficiente antes del cambio global.
Actualización de autores	Dos escrituras del frontend.	Operación atómica o recuperación explícita.
Rechazo / cancelación lista	Estado representado, pero recorridos incompletos en mock.	Acciones y tratamiento de copia según el modelo.

Tema	Estado actual	Resultado que se necesita
Reportes reales	Cálculo local sobre snapshot.	Endpoints, respuesta tipada y cambio del consumidor React.

Cómo registrar una decisión de manera útil

Anotar la situación concreta, la regla aceptada, ejemplos permitidos/prohibidos y el efecto en estados y relaciones. Después actualizar pruebas, contrato y pantalla. Evitar frases vagas como se maneja automáticamente: deben indicar qué se modifica, con qué condición y quién confirma la operación.

Código de referencia: FigJam: notas de decisiones pendientes | docs/definicion-producto.md | frontend/src/api/mock.ts

38 Guía de diagnóstico: problemas frecuentes y dónde mirar

Síntoma	Causa posible	Comprobación concreta
Pantalla vacía o error al cargar HTTP	Un recurso de snapshot no existe o su JSON no coincide.	Revisar Network, ruta fallida, status y shape; no asumir que Sedes funcionando implica todos los módulos.
No aparece un cambio de entorno	Vite no se reinició o el bundle Docker no se reconstruyó.	Ver VITE_DATA_SOURCE y reconstruir/reiniciar el proceso apropiado.
Datos desaparecen o difieren entre URLs	Mock por origen del navegador.	localhost y 127.0.0.1, o puertos distintos, no comparten necesariamente localStorage.
Un libro nuevo no tiene portada	La imagen no es un campo del modelo ni hay carga implementada.	Revisar coverByBook y el recurso visual, no inventar otro atributo sin decisión.
No se puede elegir una copia	No está disponible, libro/origen no coinciden o está comprometida.	Consultar Ejemplar, propietaria, ubicación, condición y solicitud/remito vigente.
Préstamo duplicado o doble asignación	Faltan restricciones o coordinación transaccional.	Comprobar servicio, bloqueos comunes y test con PostgreSQL.
Se pierde un autor al guardar	Falló la segunda escritura.	Comprobar respuesta de guardar Libro y PUT de autores; revisar operación atómica.
Error CSRF o 403 en POST	Sesión, token u origen no coinciden.	Ver cookie, header, inicialización CSRF, permisos y configuración del backend.
404 al recargar una ficha en producción	Servidor no aplica fallback a index.html.	Revisar Nginx try_files para rutas de React.
Compose no inicia	Falta .env, puerto ocupado o servicio con fallo.	Validar configuración y consultar logs del servicio concreto.
Reporte distinto de Pen	La maqueta usa cifras ilustrativas.	Comparar seed o datos reales con la definición de conteo del reporte.
Reporte multiplicado	Join agrega filas de autores o detalles.	Contar entidades correctas y contrastar resultados conocidos.

Buscar una operación en el código

```
REM Ejemplos de búsqueda desde la raíz, si está instalado rg
rg -n "habilitar|inhabilitar" frontend/src backend/apps
rg -n "fecha_vencimiento" frontend/src backend/apps
rg -n "asignar-reserva" frontend/src
rg -n "EN_TRANSITO|EN_TRASLADO" frontend/src
rg -n "snapshot|resourcePaths" frontend/src/api
```

Primero localizar el nombre del recurso y su acción. Luego seguir pantalla -> contrato -> adaptador -> servicio del backend -> modelos. No buscar solamente el texto del botón: statusLabels traduce etiquetas y los códigos del dominio pueden tener otros nombres visibles.

Código de referencia: frontend/src/pages/ | frontend/src/api/ | frontend/nginx.conf | docker-compose.yml

39 Glosario para comprender el código

Término	Significado en este proyecto
Componente	Función de React que representa una parte de la interfaz.
Prop	Entrada de un componente; resource configura listados y formularios genéricos.
Estado de interfaz	Dato de trabajo del navegador, como búsqueda, diálogo o mensaje.
Estado de dominio	Situación de una entidad persistente que condiciona sus operaciones.
Contexto	Acceso compartido de React a datos cargados, tema, sede y funciones.
Hook	Función de React como useState o useEffect para estado y efectos.
Adaptador	Implementación de un mismo contrato contra mock o HTTP.
DTO / payload	Estructura de datos que una petición o respuesta transporta.
FK	Referencia a otra entidad; no es copia de sus atributos.
PK / UK	Identificación primaria / unicidad de un atributo o combinación.
Serializer	Conversión y validación de entrada/salida para la API.
Servicio	Caso de uso que coordina reglas y cambios relacionados.
Selector	Consulta o cálculo de lectura sin modificar entidades.
Transacción	Unidad que confirma juntos los cambios o permite deshacerlos.
Bloqueo de fila	Coordinación de escrituras concurrentes sobre un registro compartido.
Migración	Cambio versionado del esquema que aplica el ORM.
State	Patrón que distribuye comportamiento según el estado del remito.
Strategy	Patrón que permite distintas tramitaciones bajo un contrato común.
SPA	Aplicación donde React resuelve pantallas a partir de rutas.
Snapshot	Conjunto de datos que el adaptador actual entrega al contexto.
Corte	Fecha/instante contra el que se evalúa un indicador.
Idempotencia	Evitar efectos adicionales indebidos al repetir una misma operación.

40 Mapa de fuentes y referencias para seguir estudiando

La explicación corresponde a los archivos de la copia local revisada. Los nombres relativos de este mapa se resuelven desde E:/universidad/DAO/tp-dao-G85. Las rutas HTTP marcadas propuestas aún requieren implementación y validación. El diccionario de entidades se extrajo del archivo actual de tipos.

Archivo	Qué conviene consultar
frontend/src/main.tsx	Entrada de React.
frontend/src/app/App.tsx	Todas las rutas de pantalla.
frontend/src/app/context.tsx	Carga global, tema, sede y mensajes.
frontend/src/app/Layout.tsx	Navegación, cabecera y carga/errores.
frontend/src/domain/models.ts	Tipos de todas las entidades y relaciones.
frontend/src/domain/selectors.ts	Disponibilidad, fechas, etiquetas y nombres.
frontend/src/domain/reports.ts	Criterios y cálculo de cuatro reportes.
frontend/src/domain/workflows.test.ts	Trece casos actuales de comportamiento.
frontend/src/api/contracts.ts	Interfaz del adaptador, acciones y rutas propuestas.
frontend/src/api/index.ts	Selección de mock o HTTP.
frontend/src/api/mock.ts	Persistencia de demostración y simulación de operaciones.
frontend/src/api/http.ts	Peticiones, paginación y errores.
frontend/src/data/seed.ts	Datos iniciales y portadas.
frontend/src/pages/resources.ts	Campos y títulos por recurso.
frontend/src/pages/Lists.tsx	Búsqueda, filtros, tablas y catálogo.
frontend/src/pages/EntityForm.tsx	Alta y edición.
frontend/src/pages/Details.tsx	Ficha y disponibilidad de operaciones.
frontend/src/pages/Commands.tsx	FormData, payloads y diálogos de operación.
frontend/src/pages/Other.tsx	Inicio, acceso, apariencia y devolución.

Archivo	Qué conviene consultar
frontend/src/pages/Reports.tsx	Filtros aplicados y exportación CSV.
frontend/src/components/ui.tsx	Componentes reutilizables y errores por campo.
frontend/src/styles.css	Tokens de diseño y temas.
frontend/vite.config.ts	Proxy y puerto de desarrollo.
frontend/Dockerfile	Construcción/publicación del frontend.
frontend/nginx.conf	SPA y proxy hacia Django.
backend/config/settings.py	Base de datos, apps, idioma y configuración DRF.
backend/config/urls.py	Rutas realmente registradas.
backend/apps/sedes/models.py	Modelo y unicidad existente de Sede.
backend/apps/sedes/services.py	Servicios y transacciones existentes.
backend/apps/sedes/serializers.py	Contrato y validación de Sede.
backend/apps/sedes/views.py	Listados, edición y acciones de Sede.
backend/apps/common/exceptions.py	Formato uniforme de errores.
docker-compose.yml	Servicios, puertos y volumen.
docs/definicion-producto.md	Alcance, patrones y decisiones pendientes.
docs/plan-proyecto.md	Sprints, responsabilidades y criterio de terminado.

Modelos y diseño

FigJam - Modelos de clases, estados y DER
https://www.figma.com/board/aYOy4RyT0MaeW0a69ivvxJ/Modelos

Repositorio del grupo
https://github.com/pedr0torcivia/tp-dao-G85

Pen: pen/mocks.pen define la presentación del frontend. El documento se consultó mediante sus herramientas de lectura; no se alteró el diseño para redactar la guía. Las diferencias de campos y estados de la maqueta se resolvieron a favor de las clases y el DER.

Documentación primaria consultada

[D1] Transacciones Django 5.2
https://docs.djangoproject.com/en/5.2/topics/db/transactions/

[D2] Restricciones de modelos Django 5.2
https://docs.djangoproject.com/en/5.2/ref/models/constraints/

[D3] Autenticación Django REST Framework
https://www.django-rest-framework.org/api-guide/authentication/

[D4] ViewSets y acciones DRF
https://www.django-rest-framework.org/api-guide/viewsets/

[D5] QuerySet y select_for_update Django 5.2
https://docs.djangoproject.com/en/5.2/ref/models/querysets/#select-for-update

[D6] Migraciones Django 5.2
https://docs.djangoproject.com/en/5.2/topics/migrations/

[D7] Serializers DRF
https://www.django-rest-framework.org/api-guide/serializers/

[D8] Permisos DRF
https://www.django-rest-framework.org/api-guide/permissions/

[D9] Protección CSRF Django 5.2
https://docs.djangoproject.com/en/5.2/howto/csrf/

[D10] Zonas horarias Django 5.2
https://docs.djangoproject.com/en/5.2/topics/i18n/timezones/

Consultadas el 12 de octubre de 2026. Los ejemplos de implementación son propuestas propias adaptadas al proyecto y no una copia de una aplicación genérica. Cuando una recomendación cambia el consumidor o agrega un endpoint, se identifica como trabajo futuro.

Registro de revisión de las fuentes

Se conservó un registro técnico de hashes y cantidad de líneas en los archivos intermedios de generación para identificar la versión revisada. Este registro no exige que el código esté confirmado en Git y no incluye .env ni credenciales reales.